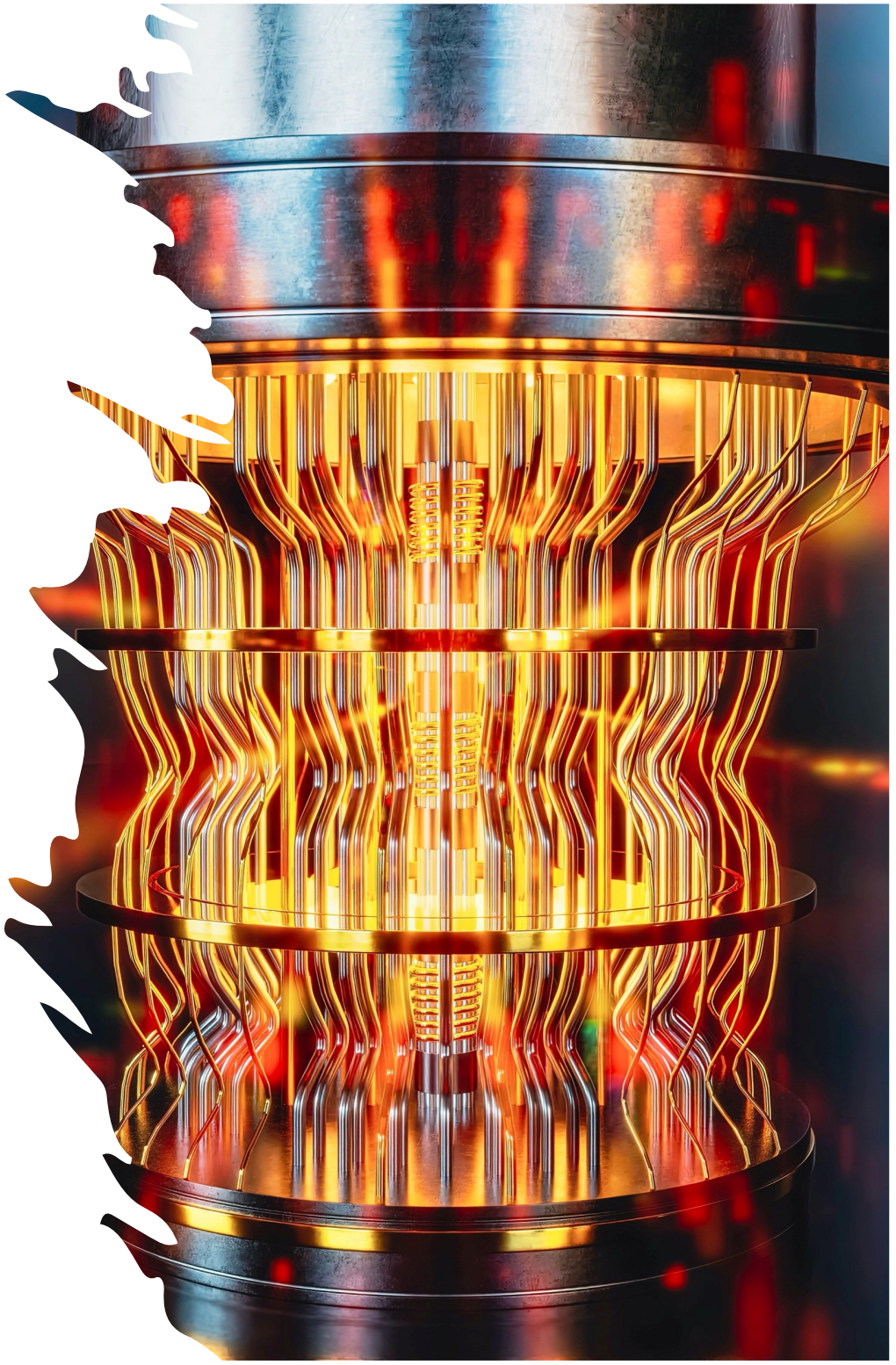
UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

Course on ***Quantum Technology for Security***

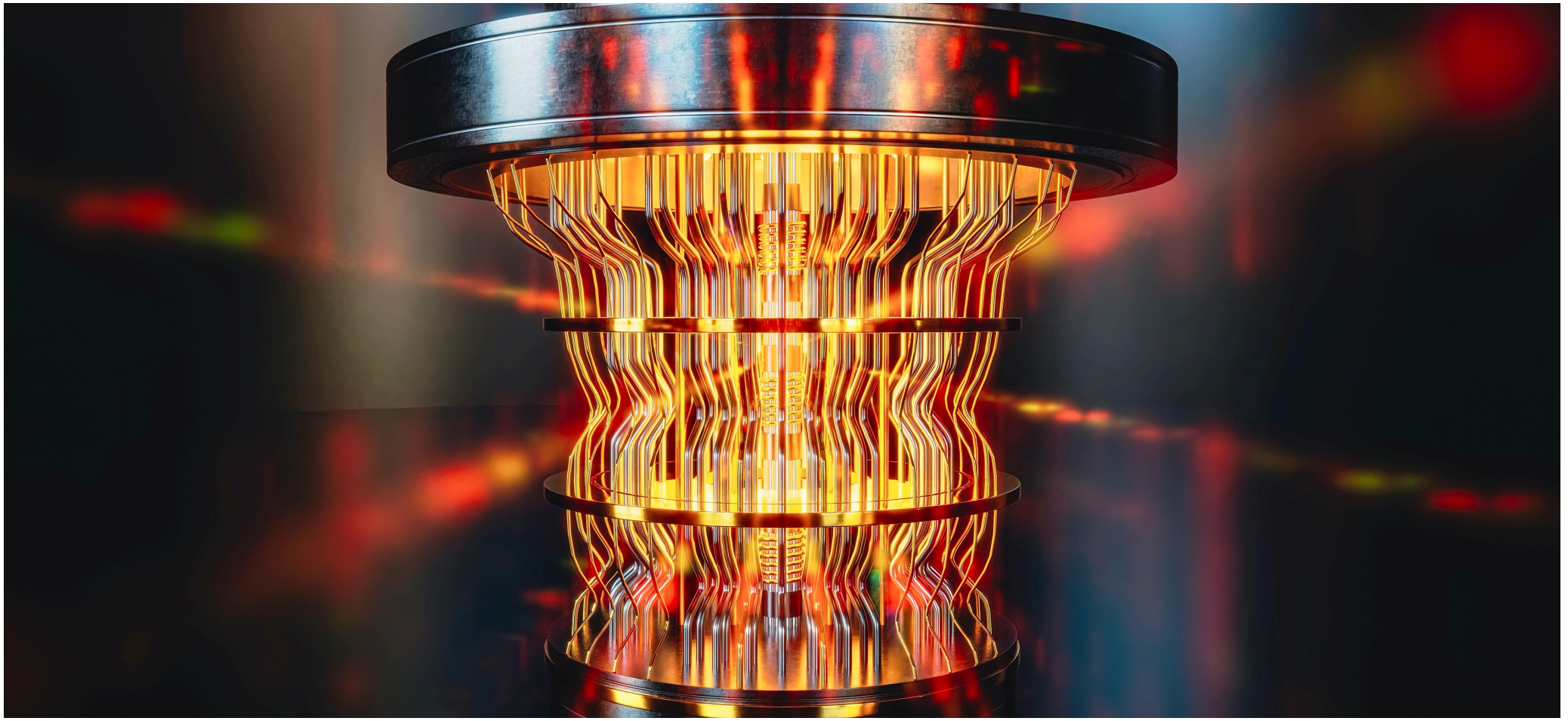
Lecture 5 – Advanced Aspects of Quantum Networking and Key Distribution



Prof. Esposito Christian

::.. Summary

- Quantum Physical Unclonable Functions (Q-PUF)
  - Introduction, possible constructions, and uses
- Quantum Random Number Generators
  - Introduction, possible constructions, and uses
- Post-Quantum Cryptography
  - Introduction, standards, and exploitations
  - Hash-, code- and lattice-based solutions



Quantum Physical Unclonable Functions

::.. Weakness in QKD

In QKD solutions such as BB84, the classical communication channel between Alice and Bob is allowed to be public, but it has to be authenticated to prevent man-in-the-middle attacks.

- Public key crypto can provide authentication. However, the aim of QKE is to achieve unconditional (information-theoretic) security, i.e. not relying on the kind of computational assumptions that public key crypto needs.

One way to achieve information-theoretic authentication is to let Alice and Bob share a short initial Message Authentication Code (MAC) key. Though this may look like cheating, it is not. QKE serves to indefinitely lengthen an initial shared secret.

::.. Weakness in QKD

In QKD solutions such as BB84, the classical communication channel between Alice and Bob is allowed to be public, but it has to be authenticated to prevent man-in-the-middle attacks.

- Public key crypto can provide authentication. However, the aim of QKE is to achieve unconditional (information-theoretic) security, i.e. not relying on the kind of computational assumptions that public key crypto needs.

Another way to achieve authentication is to let Alice and Bob each possess one part of an entangled state. This approach has a number of practical drawbacks, such as the requirement that the entangled state has to be stored for a long time.

::.. Weakness in QKD

A different solution makes use of a Hardware/Software construction cryptographic key generators to feed the classical authentication algorithm, without requiring a pre-shared digital secret key.

- **Wegman-Carter authentication** is a highly secure method for authenticating messages, as it provides unconditional security (also known as information-theoretic security).

Even an adversary with an infinite number of supercomputers or quantum computers cannot forge a message authenticated this way. Because of this "unconditional" guarantee, it is the standard method used to authenticate the classical channels in Quantum Key Distribution (QKD) protocols like BB84.

::.. Weakness in QKD

Wegman-Carter authentication relies on two mathematical building blocks:

- Universal Hashing and a One-Time Pad.

A Universal Hash Function family is a massive collection of different hash functions.

- To send a message, Alice and Bob use a shared secret key to randomly pick one specific hash function (h) out of this massive family.
- Alice passes her long message (M) through this specific function to create a short, fixed-length digest: $h(M)$.

The mathematical property of universal hashing guarantees that if an attacker modifies the message, the probability that the modified message results in the exact same digest is incredibly small, no matter how clever the attacker is.

::.. Weakness in QKD

Wegman-Carter authentication relies on two mathematical building blocks:

- Universal Hashing and a One-Time Pad.

If Alice just sends $h(M)$, an eavesdropper watching the channel could learn something about the specific hash function Alice picked. To prevent this, Alice encrypts the digest itself using a One-Time Pad (OTP).

- Alice takes a fresh, secret random string of bits (K_{OTP}) and combines it with the digest using the XOR ($\oplus$), obtaining the MAC, or "tag": $Tag = h(M) \oplus K_{OTP}$.
- Alice sends the message and the Tag to Bob.

Because it is encrypted with a One-Time Pad, the Tag looks like completely random noise to an attacker. It leaks absolutely zero information about the hash function or the message.

::.. Weakness in QKD

QKD protocols like BB84 aim to achieve absolute, unconditional security. If they used a standard classical MAC to authenticate their public channel, the overall system would no longer be unconditionally secure—it would become vulnerable to a future attacker with a powerful enough computer.

Wegman-Carter is the perfect match for QKD because it preserves that "unconditional" security promise. The only downside is that it consumes a portion of the secret key every time a message is sent. Because the One-Time Pad (K_{OTP}) part of the key can never be reused, Alice and Bob must constantly feed a fraction of their newly generated QKD keys back into the Wegman-Carter algorithm to authenticate the next round of communication.

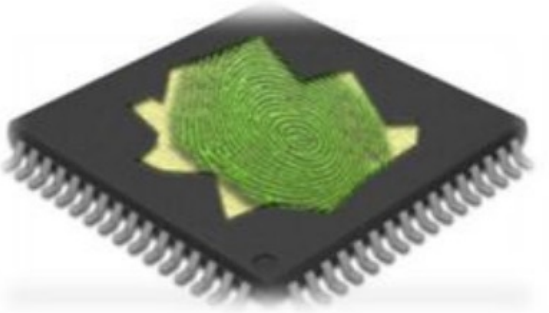
::.. Weakness in QKD

Wegman-Carter authentication suffers from a "chicken-and-egg" dilemma: to achieve unconditional security, it requires Alice and Bob to already share a perfectly secret, random digital key. If they don't have one, they cannot start the protocol.

By introducing a cryptographic key generator, the need to store a pre-shared digital key is eliminated as it manufactures the secret keys for the Wegman-Carter algorithm on demand.

Instead of pulling a static digital key out of a flash memory storage drive (which can be hacked, copied, or read via side-channel attacks), the envisioned cryptographic key generator must create the key dynamically, and the strongest possibility is to do it straight from the physical structure of the chip.

::.. Introduction



Using intrinsic random physical features to identity objects, systems and people is not new. Fingerprint identification is the key mechanism for human authentication. **Physical(ly) Unclonable Functions** (PUF) aims at realizing something similar to biometrics to IC by exploiting intrinsic electronic features of circuits.

PUF realizes a functional operation, i.e., when provided with a certain input, or challenge, it returns a given response. PUF is not a true function in the mathematical view, as multiple possible outputs corresponds to a given input.

::.. Introduction

An applied challenge and the returned response is generally called as challenge-response pair or CRP, and the relation enforced between challenges and responses by a specific PUF is called CRP behavior. The response is considered a random variable, and the distribution of its PUF responses is needed to be known. CRP behavior is implied by the PUF construction, while for others it is less obvious, and some settings must be explicitly indicated.

PUF is used in two distinct phases:

- Enrollment Phase – a number of CRP is collected from a given PUF and stored in a CRP database
- Verification phase – a randomly-selected challenge is applied to the PUF and the obtained response is compared with the corresponding response in the database.

::.. Introduction

Originally, PUF stood for physical unclonable function, but later the variant physically unclonable function also got into use. Despite being still used unchangeably, there is a small difference in meaning.

1. A physical unclonable function is ‘a physical function which is unclonable’;
2. A physically unclonable function is ‘a function which is physically unclonable’.

The second interpretation is a more fitting description of the actual concept of a PUF.

The adjective unclonable is the central specifier in the acronym, reflecting the distinguishing property of a PUF. However, the actual meaning of unclonable is not clearly defined.

::.. Introduction

It is crucial to define what a clone is:

1. Mathematical clone - any construction that accurately mimics the CRP behavior of a particular PUF could be considered a clone thereof.
2. Physical clone - any construction that has the same CRP behavior and the same physical appearance as a particular PUF, or even manufactured in the same production process, is a clone.

PUF should be made unclonable with respect to the mathematical rather than the physical definition of a clone.

A PUF is not even strictly a function, since a single challenge can be related to more than a single response due to the uncontrollable effects of the physical environment and random noise on the response generation.

::.. Introduction

A more fitting mathematical description of a PUF is as a probabilistic function, i.e., a function for which part of the input is an uncontrollable random variable.

PUFs and PUF-like constructions have been proposed based on a wide variety of technologies and materials such as glass, plastic, paper, electronic components and silicon integrated circuits (ICs). A possible classification of PUF constructions is based on the electronic nature of their identifying features.

A PUF construction is defined intrinsic PUFs, if it meets at least two conditions:

1. its evaluations are performed internally by embedded measurement equipment,
2. its random instance-specific features are implicitly introduced during its production process.

::.. Introduction

1. An external evaluation consists in the measurement of features which are externally observable and/or which is carried out using equipment which is external to the physical entity containing the features.
2. Internal evaluations implies measurements of internal features which are carried out by equipment completely embedded in the instance itself.

There are several advantages to employ internal measurements/evaluations:

1. Every PUF instance can evaluate itself without any external limitations or restrictions. Internal evaluations are typically also more accurate since there is less opportunity for outside influences and measurement errors.

::.. Introduction

2. As long as an instance does not disclose a response to the outside world, it can be considered an internal secret. This is of particular interest when the embedding object is a digital IC, since in that case the PUF response can be used immediately as a secret input for an embedded implementation of a cryptographic algorithm.

A possible disadvantage of an internal evaluation is that one needs to trust the embedded measurement equipment, since it is impossible to externally verify whether the measurement takes place as expected.

A second construction-based distinction considers the source of the randomness of the evaluated features in a PUF.

::.. Introduction

1. An explicit randomization procedure can be present within the manufacturing process of the PUF instances with the sole purpose of introducing random features which will later be measured when the PUF is evaluated.
2. The measured random features arise naturally as an artifact of uncontrollable implicit side effects during the manufacturing process.

There are some subtle but interesting advantages :

1. Implicit random variations generally come at no extra cost since they are already (unavoidably) present.
2. Implicit random variability in manufacturing is generally undesirable as manufacturers tries to reduce it as much as possible. However, completely avoiding it is technically impossible.

::.. Introduction

As a consequence, PUF constructions based on implicit process variations have the interesting security advantage that even the manufacturers cannot remove or control the random features at the core of the PUF's functionality.

A last classification is explicitly based on the security properties of their challenge-response behavior.

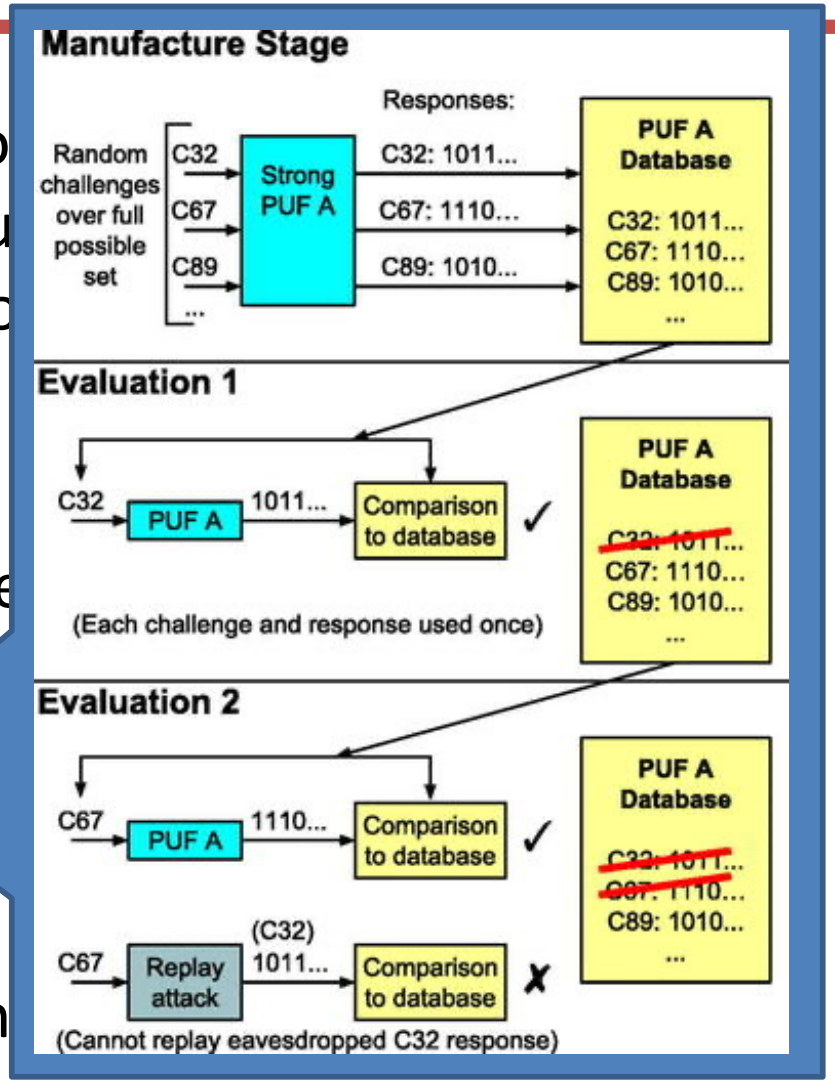
1. A PUF is called a strong if, even after giving an adversary access to a PUF instance for a prolonged period of time, it is still possible to come up with a challenge to which with high probability the adversary does not know the response.
2. PUFs which do not meet the requirements for a strong PUF, are consequentially called weak PUFs.

::.. Introduction

As a consequence, PUF constructio
variations have the interesting secu
manufacturers cannot remove or co
the core of the PUF's functionality.

A last classification is explicitly base
of their challenge-response behavi

1. A PUF is called a strong
   access to a PUF instance for a
   still possible to come up with a
   probability the adversary does n
2. PUFs which do not meet the requirements for a strong PUF, are consequentially called weak PUFs.



::.. Introduction

A strong PUF has

- a very large challenge set, since otherwise the adversary can simply query all challenges and no unknown challenges are left,
- the capability of making infeasible to built an accurate model of its observed CRP, or in other words exhibiting an unpredictable CRP behavior.

An extreme case called a Physically Obfuscated Key (POK) is a PUF construction which has only a single challenge, and represents a means to store keys inside an IC.

Constructing a practical (intrinsic) strong PUF with strong security guarantees turns out to be very difficult, up to the point where one can consider it an open problem whether this is actually possible.

::.. Introduction

All known intrinsic PUF constructions are silicon PUFs based on random process variations occurring during the manufacturing process of silicon chips.

- Delay-based silicon PUFs – They measures random variations on the delay of a digital circuit;
- Memory-based silicon PUFs – They use random parameter variations between matched silicon devices, also called device mismatch, in bistable memory elements;
- Intrinsic silicon PUF - They consist of mixed-signal circuits where an embedded analog measurement is quantized with an analog-to-digital conversion to produce a digital response representation.

::.. Introduction to Q-PUF

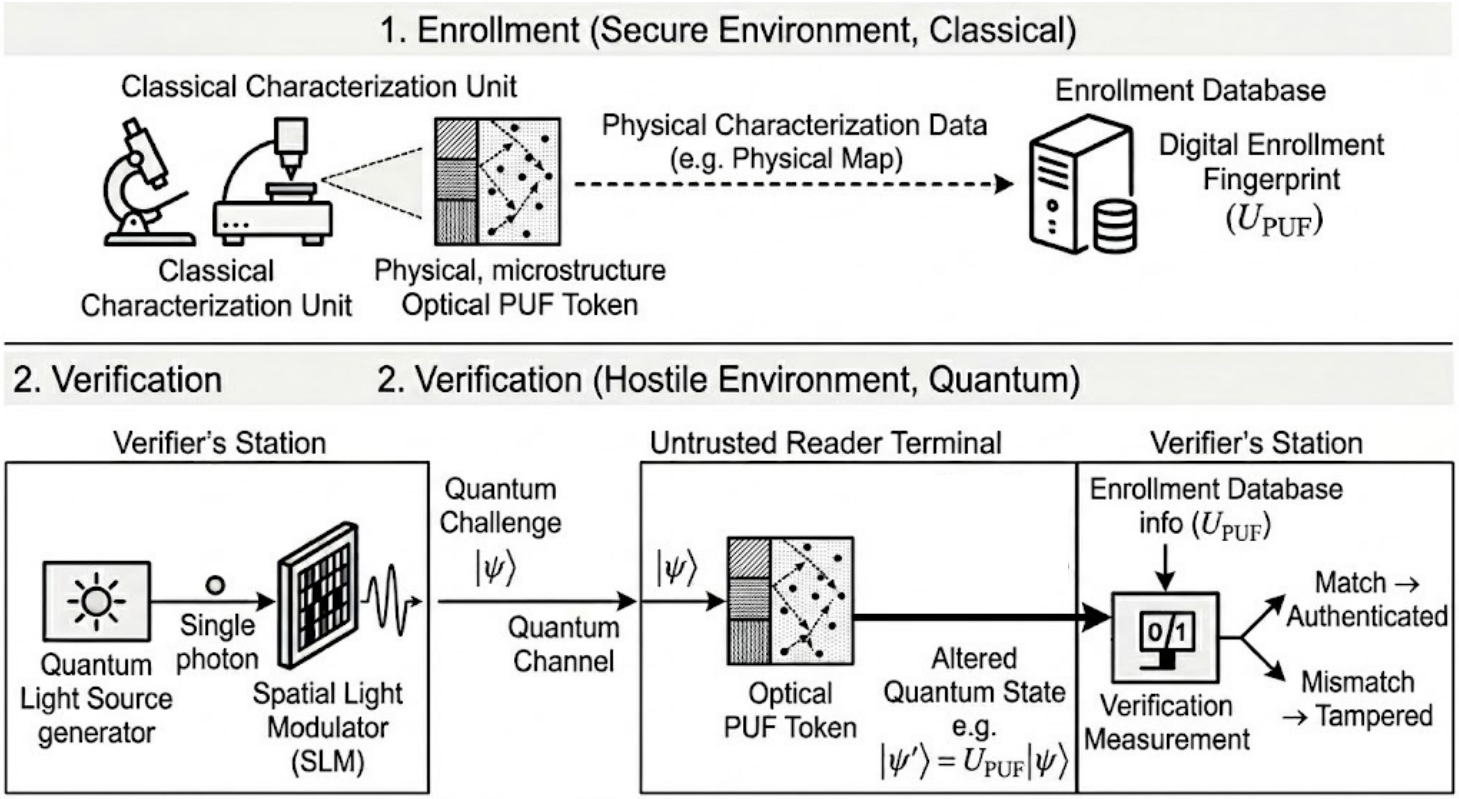
In traditional cryptography, deploying a PUF to an unsecure, remote location poses a severe vulnerability: if an adversary controls the terminal reading the PUF, they can intercept classical challenges and responses, build a digital model of the token, and easily spoof it in the future.

Quantum Readout of Physical Unclonable Functions (QR-PUF) introduces a paradigm shift by intersecting classical physical unclonability with quantum mechanics.

- The physical token remains a classical, highly complex scattering medium (such as an optical token filled with microscopic particles), but the interrogation mechanism is entirely quantum.
- Instead of a digital string, the verifier sends a fragile, unknown quantum state $|\psi\rangle$, such as a single photon with a precisely shaped wavefront, across a quantum channel.
- When this quantum challenge interacts with the PUF, it undergoes a deterministic but chaotic unitary transformation ($U_{PUF}|\psi\rangle$) dictated by the microscopic quirks of the token's structure.

::.. Introduction to Q-PUF

- The resulting state is sent back to the verifier, who uses their pre-recorded classical map of the PUF to run a projection measurement to see if the returning state matches mathematical expectations.



::.. Introduction to Q-PUF

- The resulting state is sent back to the verifier, who uses their pre-recorded classical map of the PUF to run a projection measurement to see if the returning state matches mathematical expectations.

Because of the quantum No-Cloning Theorem and the collapse of quantum states upon measurement, an adversary intercepting the challenge cannot copy it, nor can they measure it to guess the setup without introducing catastrophic, detectable noise.

By forcing the adversary into an impossible mathematical trade-off between estimation accuracy and state disturbance, QR-PUFs eliminate the need for a trusted local readout terminal, achieving a self-authenticating channel that can safely verify hardware or even bootstrap unconditionally secure Quantum Key Distribution (QKD) over unsecure networks.

::.. Introduction to Q-PUF

An alternative branch of the academic literature is to implement PUFs directly via quantum circuits, as the physical disorder moves entirely inside the quantum hardware itself. Uncontrollable manufacturing variations of semiconductor qubits and quantum logic gates are used as the source of physical entropy.

- Researchers exploit the reality that no two quantum computers are physically identical. During the nanofabrication of superconducting or silicon spin-qubit processors, microscopic imperfections inevitably occur. These variations manifest as tiny shifts in qubit frequencies, asymmetric coupling strengths between neighboring qubits, and localized cross-talk.

In a standard quantum device, these imperfections are seen as "noise" that calibration software tries to correct. In the Q-PUF literature, however, these flaws are celebrated as a unique, hardware-bound physical fingerprint.

::.. Introduction to Q-PUF

When a user runs a standardized sequence of quantum gates across these imperfect qubits, the circuit performs a chaotic, device-specific unitary transformation (U_{PUF}). The exact same quantum circuit executed on two structurally identical chips will yield entirely different quantum states due to the underlying physical variations.

The literature generally divides quantum circuit-based Q-PUFs into two main architectural frameworks:

- Inherent Silicon-Quantum PUFs: These utilize the natural physical variations of the underlying qubit hardware. For example, in charge or spin qubits, random variations in the silicon-oxide interface affect the local electrostatic environment. Passing a set of initial quantum states through a fixed array of control pulses results in an output state that uniquely identifies that specific chip.

::.. Introduction to Q-PUF

When a user runs a standardized sequence of quantum gates across these imperfect qubits, the circuit performs a chaotic, device-specific unitary transformation (U_{PUF}). The exact same quantum circuit executed on two structurally identical chips will yield entirely different quantum states due to the underlying physical variations.

The literature generally divides quantum circuit-based Q-PUFs into two main architectural frameworks:

- Personalized/Programmed Quantum PUFs: In this approach, a standard quantum register is deliberately configured or "programmed" using a digital challenge. The challenge dictates which specific qubits are coupled or which specific gate sequences are run. The physical unclonability comes from how the hardware's unique coherence times and gate errors alter that specific program. The "response" is the multi-qubit entangled state resulting from the interaction between the digital circuit design and the analog physical flaws.

::.. Introduction to Q-PUF

Moving the PUF inside the quantum circuit solves several engineering hurdles associated with optical QR-PUFs.

- It eliminates the need for bulky macro-optical components like Spatial Light Modulators (SLMs) and complex lenses, allowing the security token to be completely integrated onto a single cryogenic CMOS chip.
- It drastically reduces alignment errors, as the quantum states travel through lithographically defined waveguides or wires rather than open air or standard fiber-optic cables.

The primary security argument in this literature relies on the concept of quantum supremacy and the exponential complexity of the Hilbert space.

- For a classical circuit PUF (like an Arbiter PUF), an attacker can collect a few thousand challenge-response pairs and use classical machine learning to accurately predict the hardware's behavior.
- With a Quantum Circuit PUF, the output is a highly entangled, multi-qubit state. If the circuit utilizes n qubits, the state space scales as 2^n.

::.. Introduction to Q-PUF

For a modest number of qubits, an adversary cannot simulate or model the circuit's behavior because storing and processing a very large dimensional matrix is computationally impossible. Furthermore, because of the No-Cloning theorem, an adversary cannot intercept the output quantum state to duplicate it or study it dynamically without destroying the information.

Despite the theoretical strengths, current research is heavily focused on overcoming the "reliability vs. security" bottleneck. Because qubits are notoriously sensitive to environmental noise, temperature fluctuations and magnetic fields, a Quantum Circuit PUF can suffer from high intra-device noise. If the chip changes its behavior too much from day to day due to environmental drifts, the verifier might reject a legitimate token (a false negative). Current literature is heavily focused on developing quantum error mitigation techniques and specialized "fuzzy extractors" designed specifically for quantum states to ensure that the physical fingerprint remains stable over time without compromising its randomness.

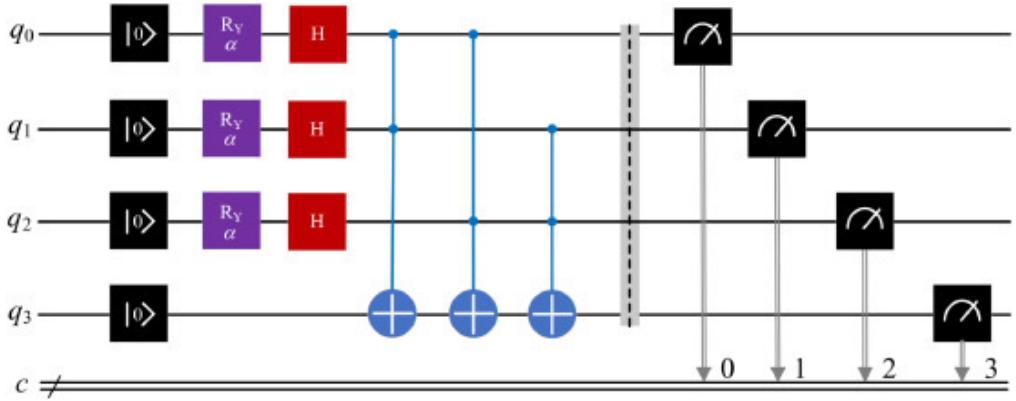
::.. Introduction to Q-PUF

The **Gate-Entanglement Q-PUF** is the most common model used when researchers evaluate Q-PUFs on public quantum computers.

- The Challenge: A specific bitstring used to initialize an array of qubits (e.g., $q_0 \rightarrow |0\rangle$, $q_1 \rightarrow |1\rangle$, $q_2 \rightarrow |1\rangle$), combined with a set of specific execution parameters (like a precise rotation angle ϑ for a phase gate).
- The Circuit Architecture:
  - Superposition: Hadamard (H) gates are applied to put the qubits into a quantum superposition.
  - Entanglement: Entangling gates like Controlled-NOT ($CNOT$) gates hook the qubits together.
  - Parameterized Rotations: Rotation gates (R_y or R_z) are applied using the angles specified by the challenge.
- The Physical Entropy Source: Due to sub-micron manufacturing variations in the superconducting loops or quantum dots, every single gate has microscopic calibration errors, cross-talk with neighboring qubits, and localized decoherence rates.

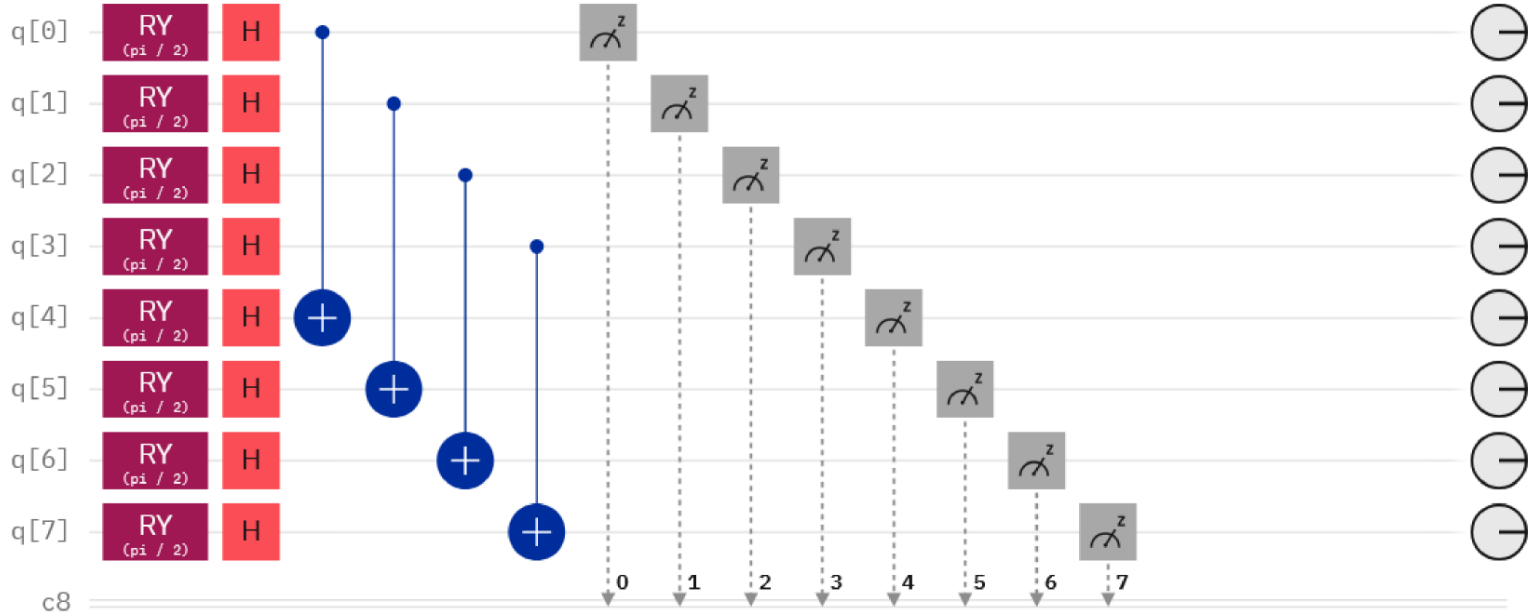
::.. Introduction to Q-PUF

- The Response (Output): When the circuit is measured over thousands of "shots," a probability distribution of bitstrings is generated. The verifier tracks the most frequently occurring outcome string. Because no two quantum chips share the exact same gate-level flaws, this most frequent string serves as a unique hardware fingerprint.



::.. Introduction to Q-PUF

- The Response (Output): When the circuit is measured over thousands of "shots," a probability distribution of bitstrings is generated. The verifier tracks the most frequently occurring outcome string. Because no two quantum chips share the exact same gate-level flaws, this most frequent string serves as a unique hardware fingerprint.



Applying R_y gate to all qubits is a deliberate, highly strategic move designed to maximize the circuit's sensitivity to hardware imperfections.

::.. Introduction to Q-PUF

- Quantum gates like the Hadamard or CNOT are mathematically fixed (e.g., exactly 90° rotations or clean bit-flips). $R_y(\theta)$ gate, however, is a parameterized rotation gate. It rotates the qubit's state vector around the Y-axis of the Bloch sphere by a continuous, variable angle θ chosen by the user (the "Challenge"). By choosing a specific angle θ, the verifier forces the quantum hardware's control electronics to generate a highly precise, analog microwave or laser pulse. No quantum hardware is perfect.

The control lines, cryogenic cables, and microwave mixers have minute, sub-micron physical variations and calibration drifts. When forced to execute a precise, fractional rotation (like $\theta = \frac{\pi}{4}$), the physical hardware will inevitably over-rotate or under-rotate by a tiny fraction ($\theta \pm \delta$).The R_y gate acts as a magnifying glass that exposes these tiny, hardware-specific pulse-calibration flaws.

::.. Introduction to Q-PUF

- If qubits are initialised to the ground state $|0\rangle$ or use a standard Hadamard gate to flip them directly to the X-axis ($|+\rangle$), the major symmetry axes of the Bloch sphere are used. Quantum computers are heavily calibrated by manufacturers to minimize errors precisely at these standard resting states.

  By applying an arbitrary $R_y(\theta)$ rotation, the qubits are intentionally kicked into an uncalibrated, intermediate state "tilt". In this territory, the qubits are vastly more vulnerable to:
  - T1 Relaxation: The natural physical decay from the excited state back to the ground state.
  - T2 Dephasing: The loss of quantum phase over time.

  Because different physical qubits on a chip decay and dephase at wildly different rates due to microscopic variations in their underlying silicon substrate, forcing them into an intermediate state causes them to accumulate these errors at completely different, unique speeds.

::.. Introduction to Q-PUF

- If the Q-PUF always started with the exact same quantum state, an attacker trying to clone the device could simply record the output multiple times and build a static lookup table. By utilizing the R_y gate with a variable angle θ, the angle itself becomes part of the cryptographic Challenge. The output state becomes a complex mathematical convolution of the user's chosen angle θ and the hardware's uncloneable physical defects. An attacker cannot easily build a machine learning model to simulate the chip because the input space is continuous, while the physical imperfections are chaotic and unpredictable.

::.. Introduction to Q-PUF

- If the Q-PUF always started with the exact same quantum state, an attacker trying to clone the device could simply record the output multiple times and build a static lookup table. By utilizing the R_y gate with a variable angle θ, the angle itself becomes part of the cryptographic Challenge. The output state becomes a complex mathematical convolution of the user's chosen angle θ and the hardware's uncloneable physical defects. An attacker cannot easily build a machine learning model to simulate the chip because the input space is continuous, while the physical imperfections are chaotic and unpredictable.

Chaotic Hamiltonian Q-PUF (The Sachdev-Ye-Kitaev Model) is a design for Q-PUFs taking advantage of chaotic quantum dynamics. The goal here is to create a circuit that acts like a "**Haar-random unitary**", which scrambles information so perfectly that it is mathematically impossible for a classical computer to guess or simulate the output.

::.. Introduction to Q-PUF

A "Haar-random unitary" is the quantum equivalent of a perfectly fair, completely unbiased coin flip or dice roll, but scaled up to the infinite world of quantum matrices.

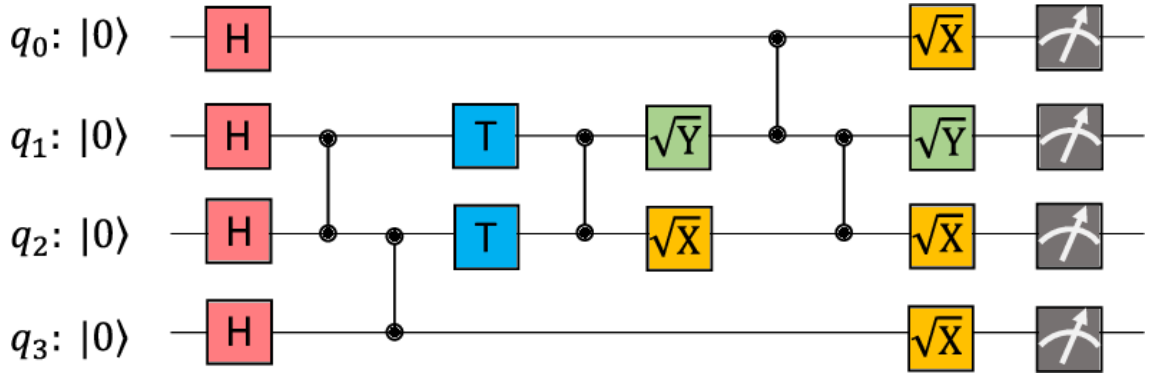
- A quantum operation is represented by a unitary matrix (U). A Haar-random unitary means you are picking a single unitary matrix out of the infinite space of all possible unitary matrices in a way that is completely uniform and unbiased.

Haar invariance: imagine to have a machine that spits out Haar-random unitary matrices. Consider one of those random matrices (U) and multiply it by a fixed, constant unitary matrix (V), the resulting matrix ($\mathrm{V} \cdot U$) is still perfectly Haar-random.

While Haar-random unitaries are beautiful in theory, they are physically impossible to implement perfectly on a real quantum computer. As the number of qubits (n) grows, the number of basic quantum gates (like Hadamards and CNOTs) required to construct a truly Haar-random unitary scales exponentially (2^n).

::.. Introduction to Q-PUF

A "Haar-random unitary" is done by constructing a Random Quantum Circuit (RQC). The general architecture follows a repeating brickwork or grid pattern across a set of n qubits.



The number of different levels of gates in between the input qubits and the output qubits are described as the depth (D) of the RQC. Going through the gates in between the input and output, the multi-qubit quantum state becomes highly entangled, which is another unique feature of quantum mechanics, leading to the exponential advantage of the quantum computing system.

::.. Introduction to Q-PUF

- The Challenge (Input): A complex, multi-qubit entangled state $|\psi\rangle$.
- The Circuit Architecture: The qubits are subjected to a highly complex, interconnected mesh of continuous quantum logic operations that simulate a heavily interacting particle system, specifically using layouts inspired by the **Sachdev-Ye-Kitaev (SYK) model**.
- The Physical Entropy Source: This model exploits the natural variations in the microscopic material substrate. In a chaotic quantum system, the final state is exceedingly sensitive to initial conditions and local variations. Even a tiny, sub-atomic manufacturing irregularity in one qubit will cause the entire multi-qubit system to scramble into a radically different state.
- The Response (Output): A highly scrambled, high-dimensional quantum state passed back to a verifier who performs a SWAP test against their enrolled digital model. An attacker trying to machine-learn this PUF would fail because classical computers cannot efficiently simulate chaotic multi-qubit quantum time-evolution.

::.. Introduction to Q-PUF

The Sachdev-Ye-Kitaev (SYK) model formulate a system where every single particle interacts with every other particle simultaneously, and the strengths of these interactions are completely random. Mathematically, the Hamiltonian (the total energy equation) of the system looks like this:

$$H = \frac{1}{4!} \sum_{i,j,k,l=1}^{N} J_{ijkl}\ \chi_i \chi_j \chi_k \chi_l$$

where χ_i represents the i-th quantum particle, where particles interact in clusters of four at a time, and J_{ijkl} is a randomly chosen coupling constant (drawn from a Gaussian distribution) among the four interacting particles.

To translate this model into a concrete quantum circuit layout, engineers face a massive topology challenge.

::.. Introduction to Q-PUF

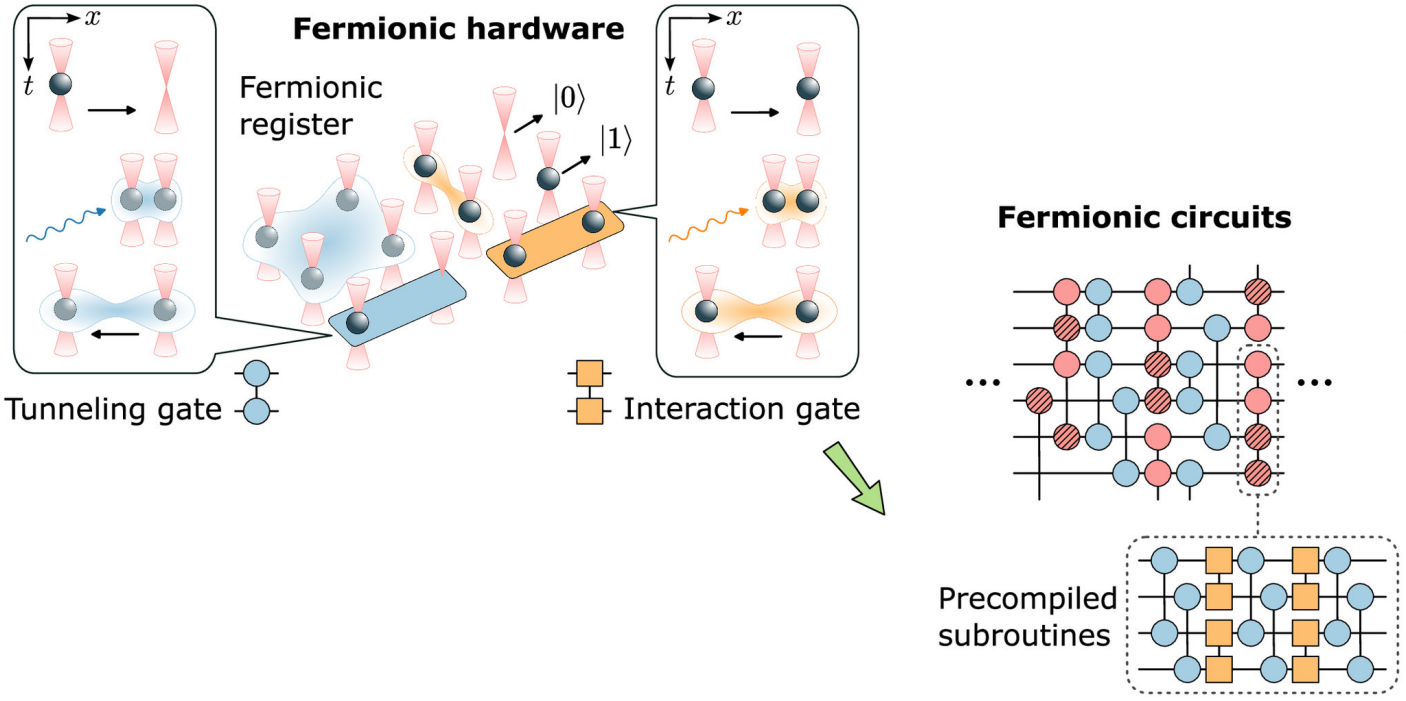
In theory, the SYK model demands all-to-all interactions, but because physical quantum processors are constrained by 2D planar geometry where qubits can usually only talk to their immediate neighbors, researchers use clever circuit architectures to mimic this all-to-all chaotic behavior.

The Fermionic Swap Network (The Planar Solution) is the most common way to implement an SYK layout on a standard 2D quantum chip. Since you cannot physically wire every qubit to every other qubit, you constantly move the quantum information around so that every particle eventually meets every other particle.

- The Geometry: Qubits are arranged in a linear chain or a tight 2D grid.
- The Cycle: The circuit alternates between layers of random 2-qubit interactions and of Fermionic SWAP (fSWAP) gates.

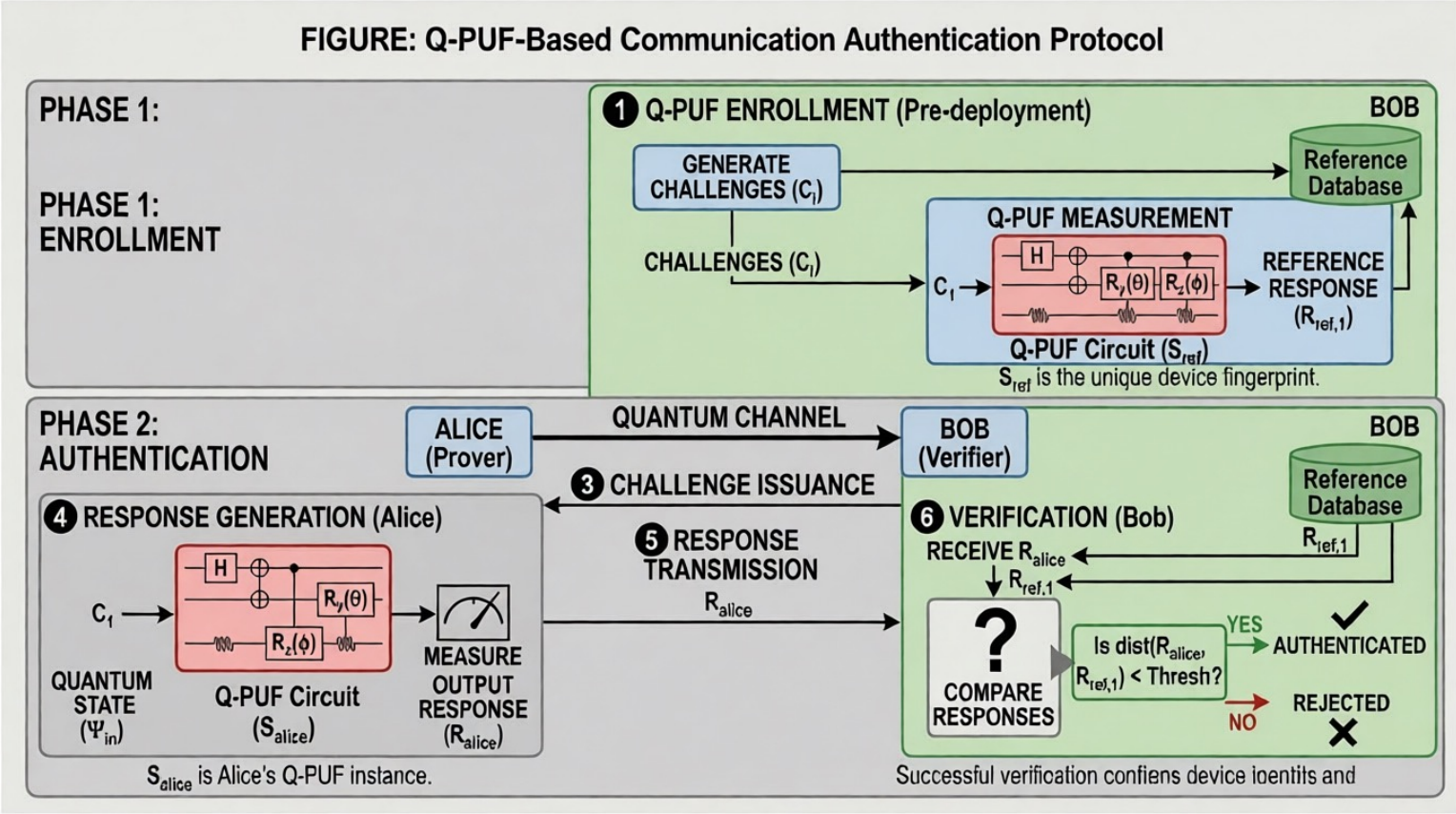
::.. Introduction to Q-PUF

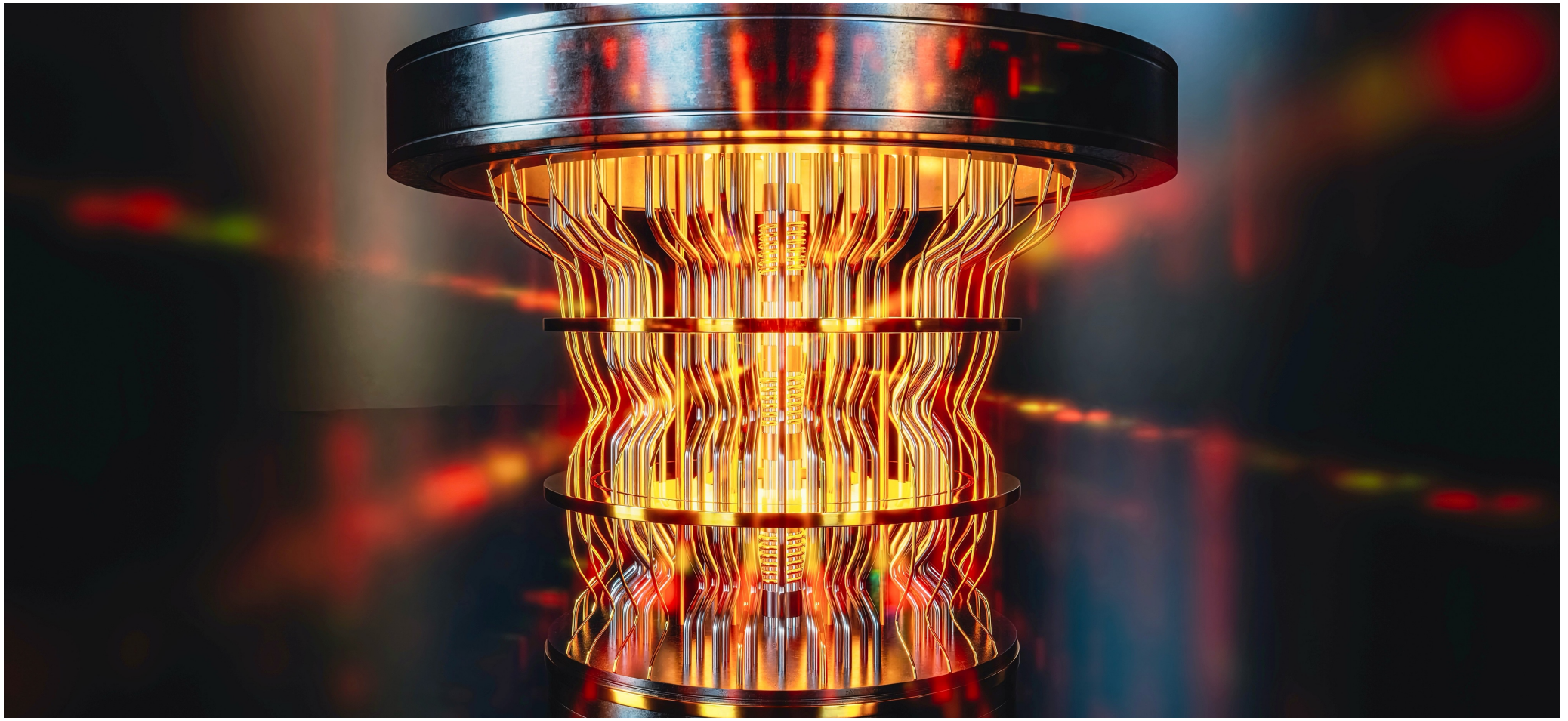
- The Result: The fSWAP gates physically shift the quantum states past each other like people dancing in a structured reel. After a specific number of steps, every single pair of qubits has sat next to each other and undergone a random interaction, successfully simulating the all-to-all requirement of the SYK Hamiltonian on a restricted physical architecture.



::.. Authentication Protocol with Q-PUF

The Q-PUF solution is used for the authentication protocol based on Challenge-response mechanism:





Quantum Random Number Generator

::.. Introduction

Randomness is a fundamental resource in modern computing.

- In classical computing, however, randomness is often only simulated. A deterministic algorithm, initialized with a finite seed, can generate a long sequence that passes many statistical tests, but such a sequence remains **pseudorandom**: in principle, anyone who knows the algorithm and the seed can reproduce the entire output.

Quantum Random Number Generation (QRNG) addresses this limitation by exploiting the intrinsic probabilistic character of quantum measurement. If a quantum system is prepared in a superposition state and then measured in a basis in which the state is not an eigenstate, the outcome cannot be predicted with certainty, even by an observer with complete knowledge of the preparation procedure.

::.. Introduction

Randomness is not merely extracted from technical noise, chaotic dynamics, thermal fluctuations, or computational complexity, but from the irreducible indeterminacy of quantum measurement.

- Suppose the qubit is initialized in the computational basis state $|0\rangle$. A Hadamard operation transforms it into the equal superposition $|+\rangle$. If the qubit is then measured in the computational basis $\{|0\rangle, |1\rangle\}$, quantum mechanics assigns probability ½ to each outcome. The result is a single random bit. Repeating the experiment produces a raw binary string.

The randomness of the resulting bit in a real experiment comes down to two profound physical realities: intrinsic indeterminism and decoherence through measurement.

::.. Introduction

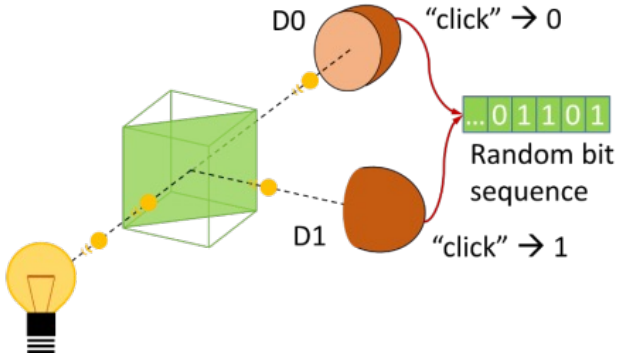
- Quantum randomness is "objective". There are no hidden variables or underlying clockwork mechanisms dictating which path the qubit will choose.
- When a physical measurement occurs, the microscopic qubit interacts with a macroscopic measuring apparatus. This interaction causes decoherence, fundamentally entangling the qubit with the trillions of atoms in the detector. During this process, the quantum superposition collapses, forcing the universe to "choose" one macroscopic reality.
- Because the initial state was perfectly balanced between $|0\rangle$ and $|1\rangle$, a real-world experiment repeated thousands of times will inevitably produce a highly uniform, unpredictable sequence of 0s and 1s, transforming pure quantum indeterminacy into a tangible, classical bit.

::.. Introduction

- In circuit notation, the elementary QRNG is

$$|0\rangle \xrightarrow{H} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \xrightarrow{\text{measure}} x \in \{0, 1\}.$$

preparation of a coherent superposition, measurement in an incompatible basis, and digitization of the outcome.



It is the quantum-circuit analogue of sending a single photon into a balanced beam splitter and assigning output port 0 or 1 depending on which detector clicks.

In both cases, ideal quantum theory predicts equiprobable outcomes, but real devices require careful modeling because detector inefficiency, bias, electronic noise, afterpulsing, loss, timing correlations, and imperfect state preparation may introduce classical structure into the output.

::.. Introduction

The security-relevant quantity is usually the conditional min-entropy,

$$H_{\min}(X \mid E) = -log_2 p_{\text{guess}}(X \mid E),$$

where X is the raw output and E denotes side information that may be available to an adversary. The value $p_{\text{guess}}(X \mid E)$ is the adversary's optimal guessing probability. A high min-entropy means that even an adversary with side information cannot predict the raw output well.

The min-entropy tells how many extractable secure random bits are actually present in the raw QRNG output. A QRNG does not usually output perfectly uniform bits directly. The raw string may be biased, correlated, partially classical, or affected by detector imperfections. Therefore, the raw output length n is not the same as the number of secure random bits.

::.. Introduction

Unlike Shannon entropy, which measures average uncertainty, min-entropy measures worst-case predictability.

A **randomness extractor** takes a raw string X, which has sufficient min-entropy but is not uniform, and maps it to a shorter string Y that is close to uniform $Y = Ext(X, S)$, where S is usually a short random seed, depending on the extractor construction. Universal hashing, including Toeplitz hashing, is widely used because it has strong mathematical guarantees. The central result behind this is the Leftover Hash Lemma. Informally, it says that if the raw string has at least k bits of min-entropy, then one can extract approximately $m \lesssim k - 2\log_2\left(\frac{1}{\varepsilon}\right)$ nearly uniform bits, where ε is the desired security distance from ideal uniform randomness. Thus, min-entropy determines the safe compression rate.

::.. QRNG Solutions

- The most basic solutions are called trusted-device QRNGs, where the user trusts the physical description of the source and measurement apparatus. Existing examples can reach very high rates and are attractive for practical deployment, but their security depends on an accurate model of the devices.
  - If one trusts the source and detectors, entropy estimation can be efficient and the extraction rate can be high.
  - A mismatch between model and implementation may compromise the output.

  Thus, practical QRNG design must distinguish between quantum entropy and classical technical noise, which is dangerous to count it as secure entropy unless it is properly bounded and not controlled/predicted by an adversary.

::.. QRNG Solutions

- To reduce the required trust in the hardware, several stronger QRNG security models have been developed.
  - Source-independent QRNG assumes that the measurement device is trusted but the source may be untrusted or uncharacterized. Randomness is certified from measurements even when the incoming states are not fully modeled.
  - Measurement-device-independent QRNG takes a complementary approach. Here, the preparation side is trusted or characterized, while the measurement device is treated as untrusted or partially untrusted.
- The strongest conceptual model is device-independent QRNG, where one does not need to trust the internal functioning of the devices.

::.. QRNG Solutions

Randomness is certified from the observed violation of a Bell inequality, as their correlations cannot be explained by any local hidden-variable model.

- If two spatially separated measurement devices produce outputs whose correlations violate a Bell inequality under appropriate assumptions, then the outcomes cannot have been predetermined in a classical local way. This makes Bell nonlocality a certificate of randomness.

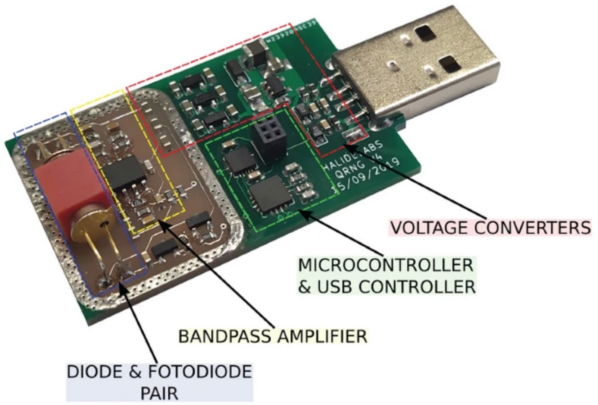
In Bell-based QRNG, the typical setup uses pairs of entangled particles. Alice and Bob independently choose measurement settings and record outcomes. A Bell parameter, such as the CHSH value, is estimated from the observed correlations.

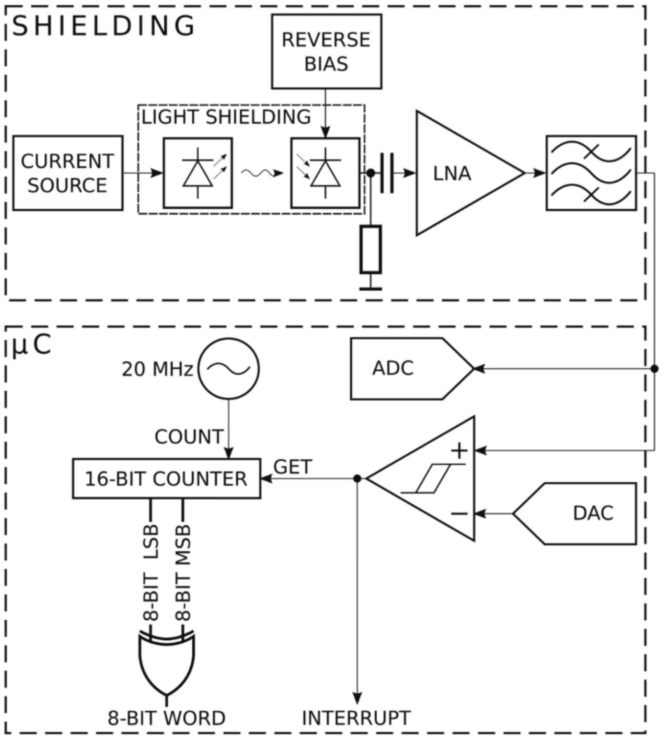
::.. QRNG Solutions

If the Bell violation is statistically significant and the relevant loopholes are closed, one can derive a lower bound on the output min-entropy. The essential point is that the randomness is certified by correlations, not by a detailed internal model of the apparatus. This is why Bell-based QRNG is called device-independent. It is, however, experimentally demanding. One must address detection efficiency, locality, freedom of choice, finite statistics, memory effects, and possible side channels. High-quality entangled sources, fast random basis selection, efficient detectors, and careful space-time separation are required.

::.. QRNG Product

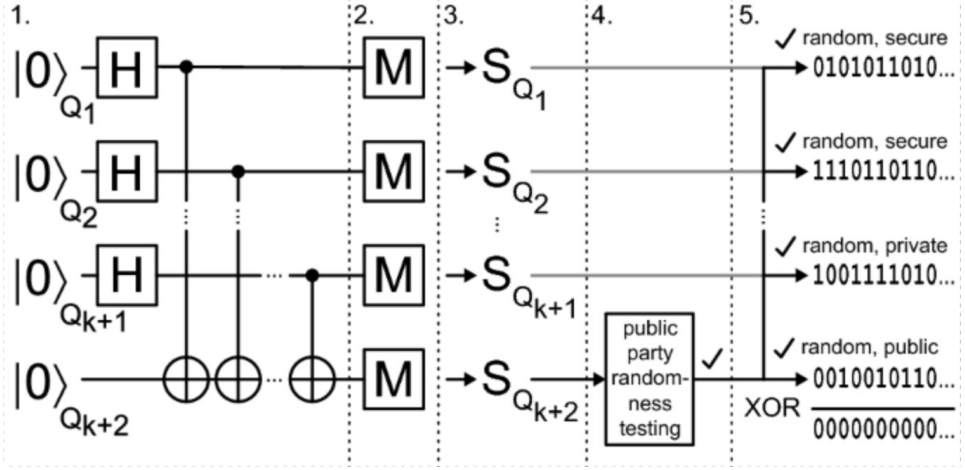
The JUR02 QRNG is an implementation of a QRNG and implements a complete generation pipeline combining a quantum entropy source, digitization, post-processing, and certification/validation procedures consistent with the broader QRNG literature on quantum unpredictability and extractor-based output conditioning.

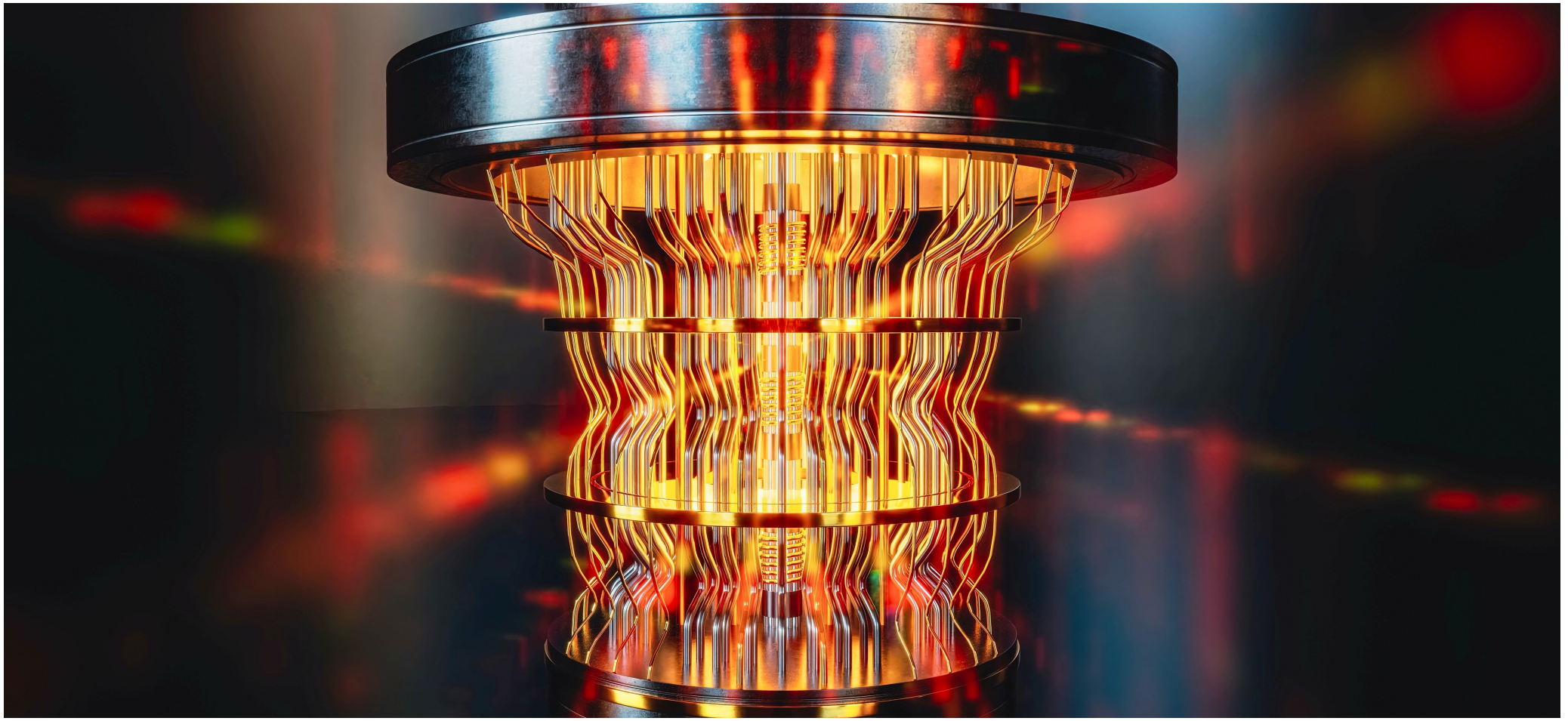




::.. QRNG Product

The protocol of the multiqubit, entanglement based, cryptographically secure QRNG with public randomness verification prepares a multiqubit entangled state, distributes the qubits to separated measurement modules, and measures them in randomly selected bases so that the observed correlations can be tested against a Bell-type or entanglement-witness criterion. The raw outcomes are accepted only if the public verification test certifies sufficient quantum entropy; then a randomness extractor compresses the accepted data into a shorter cryptographically secure bitstring whose uniformity and unpredictability are guaranteed against an adversary.

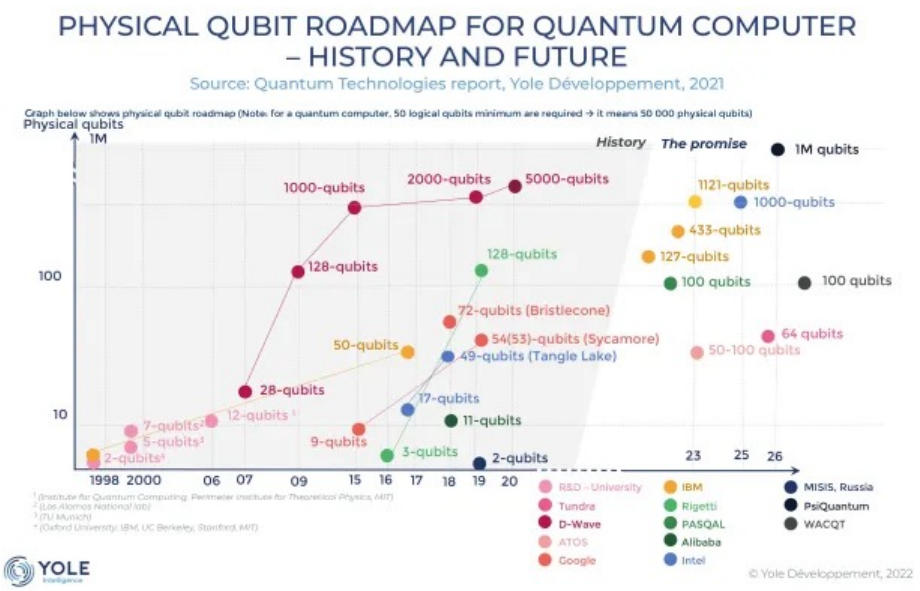


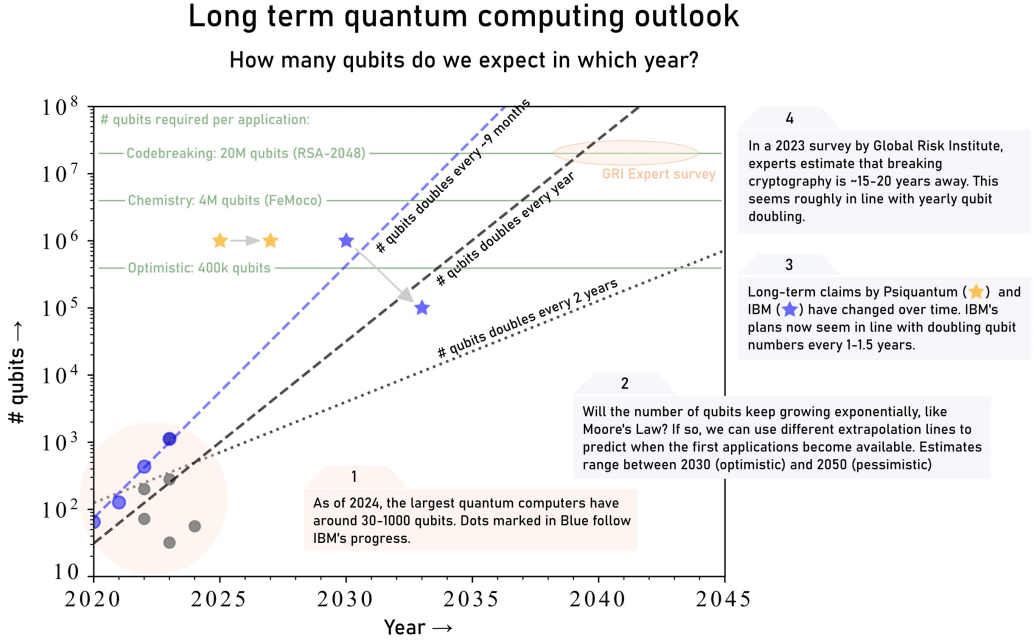


Post-Quantum Cryptography

::.. Introduction

Large-scale Quantum Computers do not exist! But they might exist in 10-20 years.





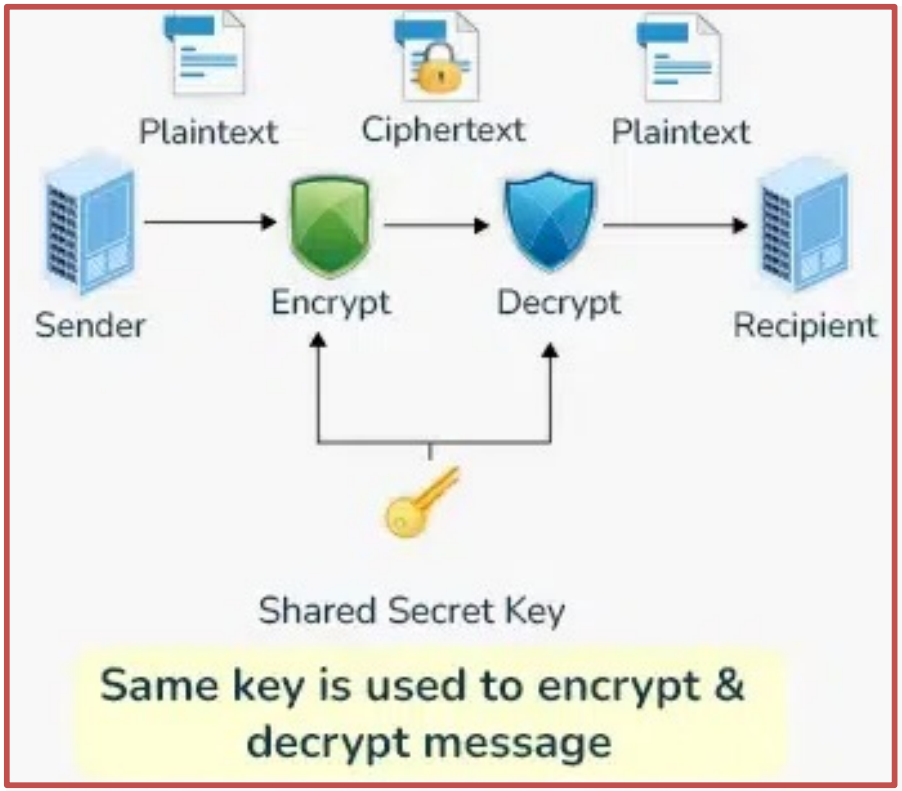
::.. Introduction

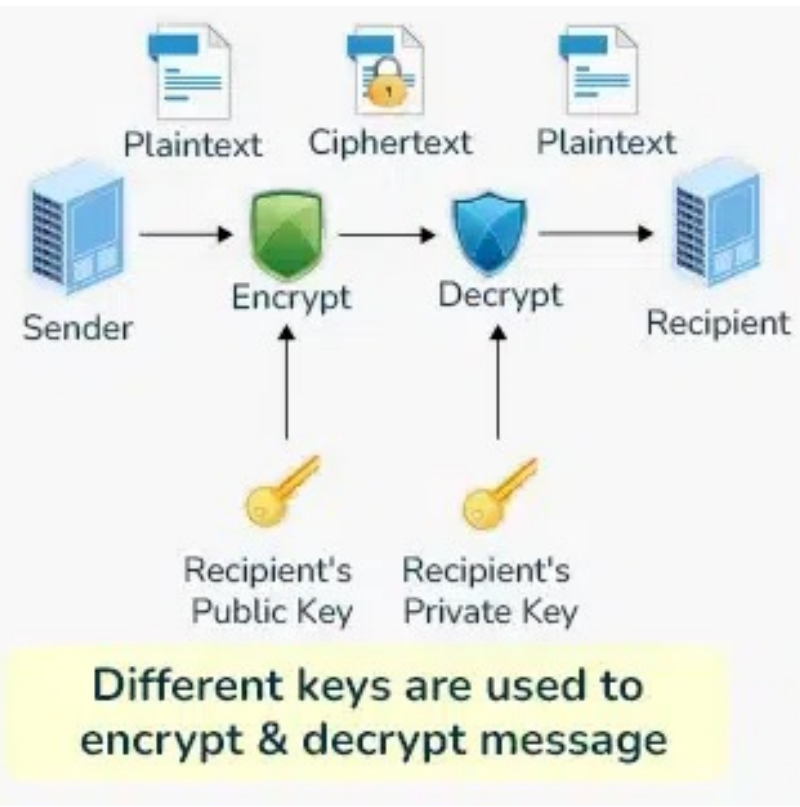
Large-scale Quantum Computers do not exist! But they might exist in 10-20 years.

Cryptography is divided into two families:

::.. Introduction

For a symmetric cipher such as AES with an n-bit key, the best generic classical attack is exhaustive search 2^n. A large-scale quantum computer can apply Grover's algorithm, reducing exhaustive key search to roughly $\sqrt{2^n} = 2^{\frac{n}{2}}$. So AES-128 would offer only about 2^{64} quantum search complexity, while AES-256 still gives about 2^{128} post-quantum security. Hence, symmetric cryptography is not destroyed; one can often compensate by increasing key sizes, and this is the reason AES-256 has been designed for.

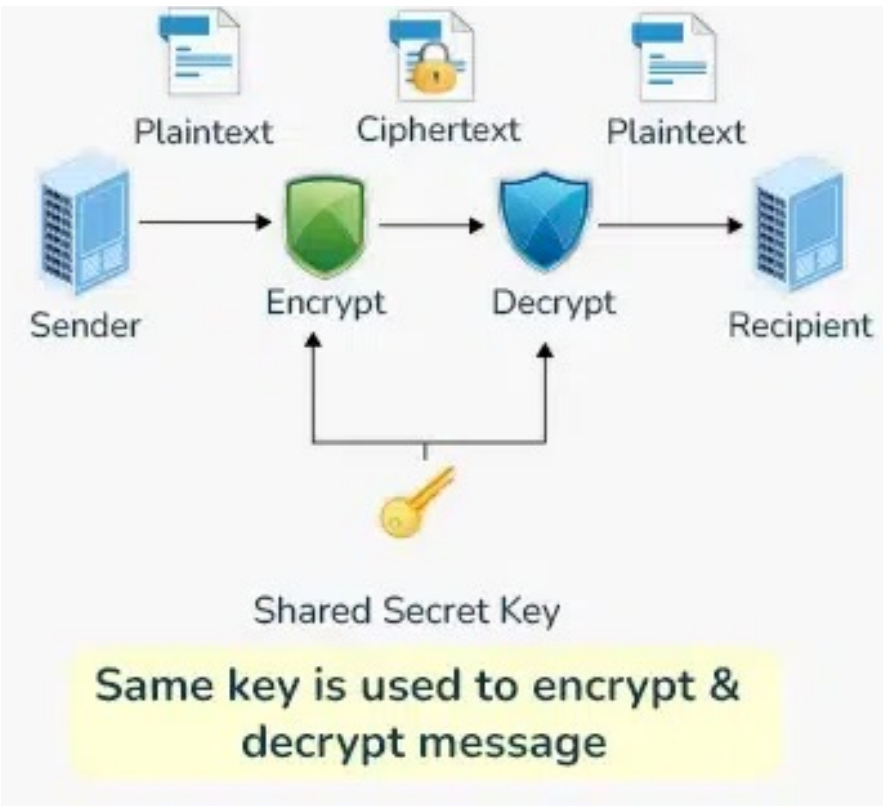


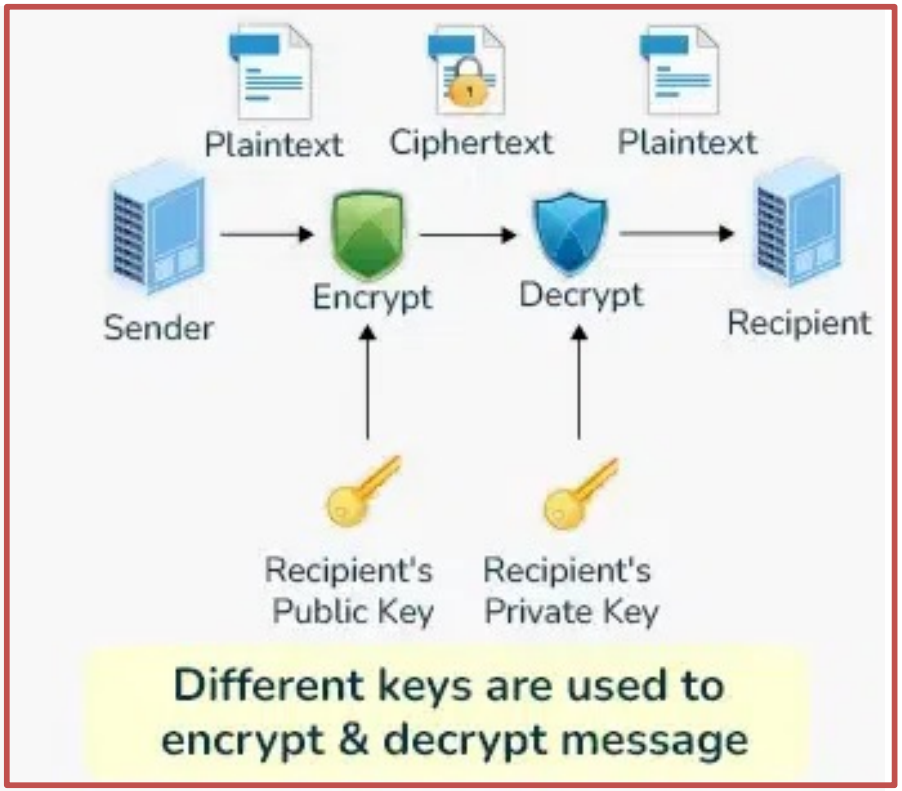


::.. Introduction

Public-key cryptography is much more seriously affected. The dominant deployed families are based on: 1. RSA integer factorization, 2. Diffie–Hellman / DSAdiscrete logarithms, 3. ECC / ECDSA / ECDHelliptic-curve discrete logarithms.

A sufficiently large quantum computer running Shor's algorithm can solve integer factorization and discrete logarithms efficiently. Therefore, RSA, classical Diffie--Hellman, DSA, ECDSA, and ECDH are considered broken in the large-scale quantum-computer model.

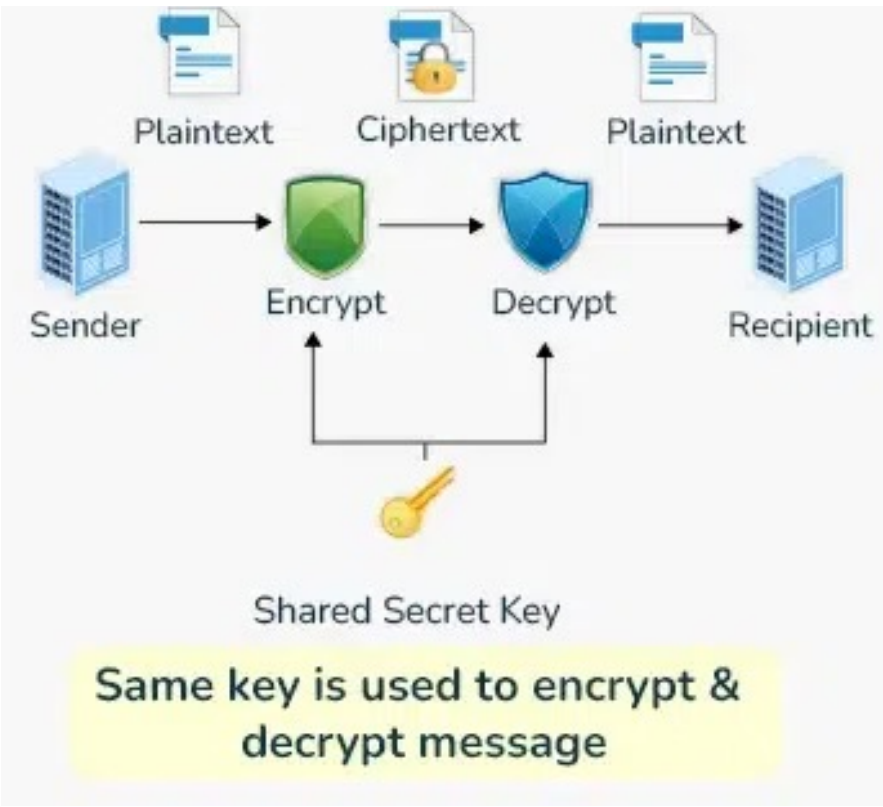


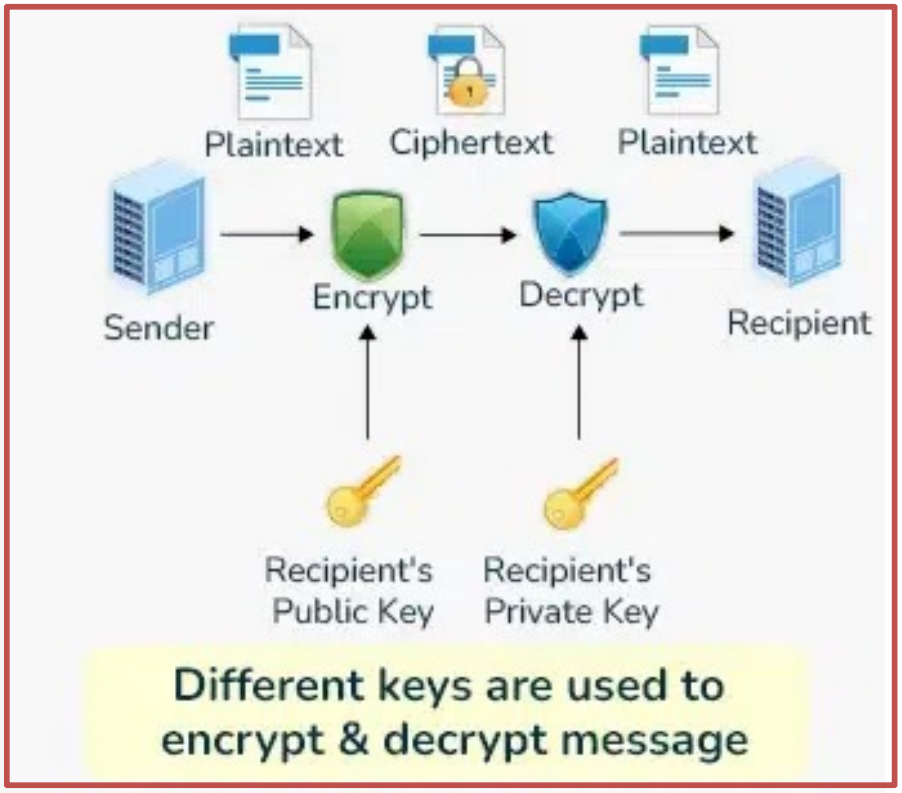


::.. Introduction

Post-Quantum Cryptography (PQC) refers to public-key cryptography not based on factoring or discrete logarithms but on a more complex mathematical problem, so that even with a large-scale quantum computer, it remains untreatable. The major practical post-quantum families are hash-based, code-based, lattice-based, multivariate, or Isogeny-based.

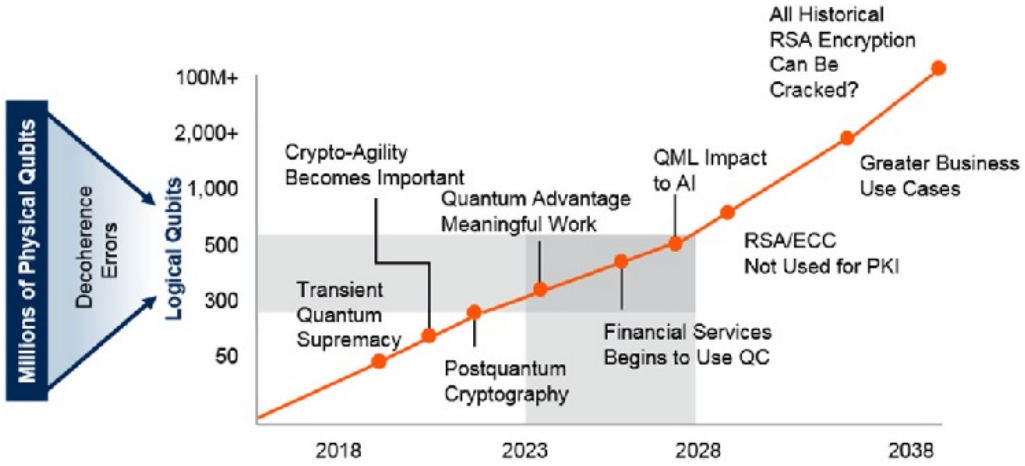
Hash-based cryptography is especially conservative because it relies mostly on the assumed one-wayness and collision resistance of hash functions.





::.. Introduction

The **NIST Post-Quantum Cryptography** timeline is now in its implementation and expansion phase. The finalized core standards (FIPS 203, 204, and 205) are published, with secondary algorithms advancing through evaluations and a strict 2030 federal migration deadline approaching.



January 2019:
Second-round candidates announced. NISTIR 8240, Status Report on the First Round of the NIST Post-Quantum Cryptography Standardization Process, released. The public comment period on the second-round candidates began

October 2021:
NCCoE Final NCCoE project description "Migration to Post-Quantum Cryptography" and the Federal Register Notice soliciting industry collaborators

December 2023:
NIST Special Publication 1800-38B, Quantum Readiness

September 2024:
Continued inclusion of completed FIPS in national and international Standards Bodies

2019 — 2021 — 2023 — 2024

2016 — 2020 — 2022 — 2024

December 2016:
Federal Register Notice – Announcing Request for Nominations for Public-Key Post-Quantum Cryptographic Algorithms

October 2020:
NIST hosts Virtual Workshop on Considerations in Migrating to PQC Algorithms at the NCCoE

July 2022:
Candidate algorithms to be standardized are announced, along with alternate candidates advancing to the fourth round. NIST-IR 8413-upd1, Status Report on the Third Round of the NIST Post Quantum Cryptography Standardization Process, released.

July 2024:
Publication of Final first three PQC FIPS

::.. Digital Signatures

A digital signature scheme has three algorithms:

- $KeyGen() \rightarrow (sk, pk)$,
- $Sign_{sk}(M) \rightarrow \sigma$,
- $Verify_{pk}(M, \sigma) \rightarrow \{true, false\}$.

Alice owns a private signing key sk and publishes a public verification key pk.

- To sign a message M, she computes $\sigma = Sign_{sk}(M)$.
- Bob checks $Verify_{pk}(M, \sigma)$.

A secure signature gives three main properties:

- sender authenticity, because only Alice should be able to produce valid signatures under Alice's public key;
- message integrity, because modifying M should invalidate the signature;

::.. Digital Signatures

- non-repudiation, because Alice should not later be able to plausibly deny having signed M, assuming her private key was not compromised and her public key is authenticated.

The last assumption matters: Bob must really possess Alice's public key. In practice, this is handled using certificates, PKI, fingerprints, or other authentication mechanisms.

::.. Naive Hash-based Signature

Let $F: \{0,1\}^n \rightarrow \{0,1\}^n$ be a one-way function, usually instantiated by a cryptographic hash function or a hash-based compression construction.

Alice chooses a random secret value $\{0,1\}^n \xrightarrow{\$} r$, which is her private key, $sk = r$. Her public key is $pk = F(r)$.

- To sign, the naive idea is simply to reveal r: $\sigma = r$.
- Verification checks whether $F(\sigma) = pk$.

This gives a primitive form of sender authenticity: if Bob knows that $pk = F(r)$ is really Alice's public key, then anyone producing r must have known the preimage of pk.

However, this is not a secure digital signature scheme, because the message M is not involved.

::.. Naive Hash-based Signature

The signature is the same for every message $\sigma = r$ regardless of M. Therefore an attacker can replace (M, σ) by (M', σ) and verification still succeeds. Thus:

- authenticity: partially yes,
- integrity: no,
- non-repudiation: no.

::.. Hash Chains

A central tool is the iterated application of a one-way function.

Define $F^0(x) = x$, $F^1(x) = F(x)$, $F^2(x) = F(F(x))$, and generally $F^i(x) = \underbrace{F(F(\cdots F(x)\cdots))}_{i\ times}$ is a **chain of hashes**.

Because F is one-way, given $F^i(x)$, one can efficiently compute $F^{i+1}(x)$, $F^{i+2}(x)$, … but should not be able to compute earlier chain values such as $F^{i-1}(x)$.

So, movement along the chain is easy only in the forward direction.

::.. Winternitz One-time Signature

Suppose messages are only $w = 3$ bits long. Then, a message represents an integer $m \in \{0,1, \dots, 7\}$.

Alice chooses $\{0,1\}^n \xrightarrow{\$} r$ so that her private key is $sk = r$ and her public key is the end of a hash chain of length 7: $pk = F^7(r)$.

- To sign message m, Alice reveals the chain value at position m:
- $\sigma = F^m(r)$.
- To verify, Bob hashes the signature forward until the end of the chain: $F^{7-m}(\sigma) =? pk$. Indeed, $F^{7-m}(\sigma) = F^{7-m}(F^m(r)) = F^7(r) = pk$.

This is the basic idea behind **Winternitz one-time signatures**: the message determines how far along a hash chain the signer reveals a value.

::.. Winternitz One-time Signature

The above scheme is still insecure.

Suppose Alice signs $m = 5$ by sending $\sigma = F^5(r)$.

An attacker Oscar can compute $\sigma' = F(\sigma) = F^6(r)$. Then σ' is a valid signature for $m' = 6$.

Because Bob verifies $F^{7-6}(\sigma') = F^1(F^6(r)) = F^7(r) = pk$.

So the attacker cannot decrease the message value, because that would require inverting F, but he can increase it $5 \rightarrow 6$, $5 \rightarrow 7$, or more generally $m \rightarrow m'$ for $m' > m$.

This is the **incrementation attack**. The problem is that hash chains are one-way only in one direction. Once a chain value is revealed, anyone can move it forward.

::.. Winternitz One-time Signature

Now consider a real message. Let the message digest to be signed have length N bits. In practice one usually signs a hash of the original message: $D = H(M)$, where $D \in \{0,1\}^N$.

Choose a Winternitz parameter w, meaning that chunks have w bits. Let $b = 2^w$, and each w-bit chunk represents a digit in base b: $a_i \in \{0,1,\dots,b-1\}$, we can split the N-bit digest into $\ell_1 = \left\lceil \frac{N}{w} \right\rceil$ digits: $D = (a_0, a_1, \dots, a_{\ell_1 - 1})$.

Each digit a_i will be signed using one independent hash chain.

To prevent the incrementation attack, Winternitz signatures append a checksum.

::.. Winternitz One-time Signature

Define $C = \sum_{i=0}^{\ell_1 - 1} ((b-1) - a_1)$, if the attacker increases some message digit a_i, then the checksum value C decreases.

Represent C in base b using $\ell 2 = \lceil \log_b(\ell_1(b-1)+1) \rceil$ digits: $C = \left(a_{\ell_1}, a_{\ell_{1+1}}, \ldots, a_{\ell_1+\ell_2-1}\right)_b$.

Now define the full digit sequence $A = (a_0, a_1, \ldots, a_{\ell-1})$, where $\ell = \ell_1 + \ell_2$, where the first ℓ_1 digits encode the message digest; the last ℓ_2 digits encode the checksum. The checksum is what prevents the incrementation attack: increasing a message digit forces the checksum integer to decrease, and decreasing at least one checksum digit requires moving backward in a hash chain, which should be infeasible.

::.. Winternitz One-time Signature

WOTS key generation - For every digit position $i \in \{0, \dots, \ell - 1\}$,

Alice chooses an independent random secret value $\{0,1\}^n \xrightarrow{\$} x_i$.

The private key is $sk = (x_0, x_1, \dots, x_{\ell-1})$.

For each x_i, compute the end of a hash chain of length $b - 1$: $y_i = F^{b-1}(x_i)$. The public key is $pk = (y_0, y_1, \dots, y_{\ell-1})$.

So each public-key component is the endpoint of one hash chain.

WOTS signing - To sign message M, first compute $D = H(M)$.

Split D into base-b digits: $(a_0, a_1, \dots, a_{\ell_1-1})$.

Compute the checksum: $C = \sum_{i=0}^{\ell_1-1} ((b-1)-a_i)$.

Encode C as ℓ_2 base-b digits and append them: $A = (a_0, a_1, \dots, a_{\ell-1})$.

::.. Winternitz One-time Signature

Then the signature consists of one hash-chain value per digit: $\sigma_i = F^{a_i}(x_i)$. Thus, the full signature is $\sigma = (\sigma_0, \sigma_1, \dots, \sigma_{\ell-1})$. Each component reveals the chain position corresponding to the signed digit.

WOTS verification - Given M, σ, pk, Bob recomputes $D = H(M)$, splits it into digits, recomputes the checksum, and obtains the full digit sequence $\mathrm{A} = (a_0, a_1, \dots, a_{\ell-1})$.

For each position i, he checks whether hashing the signature component forward to the end of the chain gives the public key component: $F^{b-1-a_i}(\sigma_i) =? y_i$. Since $\sigma_i = F^{a_i}(x_i)$, verification succeeds because $F^{b-1-a_i}(\sigma_i) = F^{b-1-a_i}(F^{a_i}(x_i)) = F^{b-1}(x_i) = y_i$.

::.. Winternitz One-time Signature

The full verification condition is $\forall i \in \{0, \ldots, \ell - 1\} : F^{b-1-a_i}(\sigma_i) = y_i$. If every equality holds, the signature is accepted.

Without a checksum, the attacker could increase message digits. With the checksum, producing a valid signature for that decreased checksum digit would require moving backward along a hash chain, which is infeasible.

Thus the checksum converts the attacker's ability to move forward into a useless capability: any valid forgery would require at least one backward move.

::.. Winternitz One-time Signature

The security of WOTS rests mainly on the one-wayness of F.

A valid signature reveals some intermediate values. Because F is efficiently computable, those revealed values allow forward movement along chains. Therefore, WOTS is safe only if each key pair is used once.

The two central dangers are:

- inverting F, which would allow computing earlier chain values;
- reusing a WOTS key, which reveals multiple intermediate points on the same chains and can enable forgeries.

That is why it is called a one-time signature scheme.

::.. Winternitz One-time Signature

A WOTS public key $pk = (y_0, \ldots, y_{\ell-1})$ must be used for only one message.

If Alice signs two different messages using the same secret key, she reveals two chain values per affected position: $F^{a_i}(x_i)$ and $F^{a_i'}(x_i)$.

The attacker can keep the earlier of the two and hash forward from it, gaining the ability to sign a range of digit values. With enough reused signatures, the attacker learns enough intermediate chain values to forge signatures for new messages.

So WOTS by itself is not a many-time signature scheme.

::.. Winternitz One-time Signature

To build practical many-time hash-based signatures, WOTS is combined with a hash tree, usually a Merkle tree. The Merkle tree authenticates many one-time public keys using one compact global public key.

::.. Winternitz One-time Signature

Modern schemes usually do not use bare WOTS. They use WOTS+, which adds masks/bitmasks and domain separation to improve provable security.

- Instead of simple chains $F(F(F(x)))$, WOTS+ uses keyed or masked chaining functions, roughly of the form $x_{i,j+1} = F(x_{i,j} \oplus r_{i,j})$, or similar variants depending on the specification.

The point is to obtain tighter security reductions from standard hash-function assumptions rather than relying on an idealized plain hash chain.

Real hash-based signatures use many calls to hash functions for different purposes. These calls must not be confused with each other. Implementations therefore use domain separation, addresses, prefixes, keys, or function identifiers.

::.. Winternitz One-time Signature

For example, hashing a Merkle-tree internal node should be distinguishable from hashing a WOTS chain step. Often all are built from the same underlying hash primitive, but with different prefixes or address values.

Modern hash-based signatures usually do not sign just $H(M)$.

They often use randomized hashing $D = H(R \parallel M)$, where R is derived from secret or pseudorandom material.

This also implements domain separators: prepending a unique, application-specific context string or prefix to the raw data before it is hashed: $D = H(R \parallel M)$, where R is the application-specific context string.

::.. Winternitz One-time Signature

By embedding a unique identifier into the input, it prevents cross-protocol attacks, stops accidental collisions between different data types, and keeps multi-purpose cryptographic states isolated. Reusing the same hash function across multiple contexts creates significant security vulnerabilities:

- Cross-Protocol Attacks: An attacker might take a valid signature generated for one protocol and trick a different application into accepting it as valid.
- Collisions & Mix-ups: Hashing different types of data (like a user ID vs. a balance) might yield the exact same hash, allowing manipulation of data structures like Merkle Trees.
- Reversible Contexts: An attacker could exploit identical hashing processes to translate inputs across different environments.

::.. Winternitz One-time Signature

Hash-based signatures have a nice one-way chain property. Revealing $F^a(x)$ does not reveal x or earlier values $F^{a-1}(x)$, $F^{a-2}(x)$, … assuming F is one-way. This makes them conceptually robust. But the one-time restriction is absolute: revealing multiple chain positions from the same secret chain gives the attacker more power.

Hash-based signatures can replace ECDSA or RSA signatures, but they cannot replace ECDH or RSA encryption. For post-quantum key exchange / encryption, one normally looks at KEMs such as lattice-based or code-based schemes.

Hash-based signatures are attractive because their assumptions are conservative: security mostly relies on hash functions.

::.. Winternitz One-time Signature

They are also conceptually simple and well-studied. But they have disadvantages:

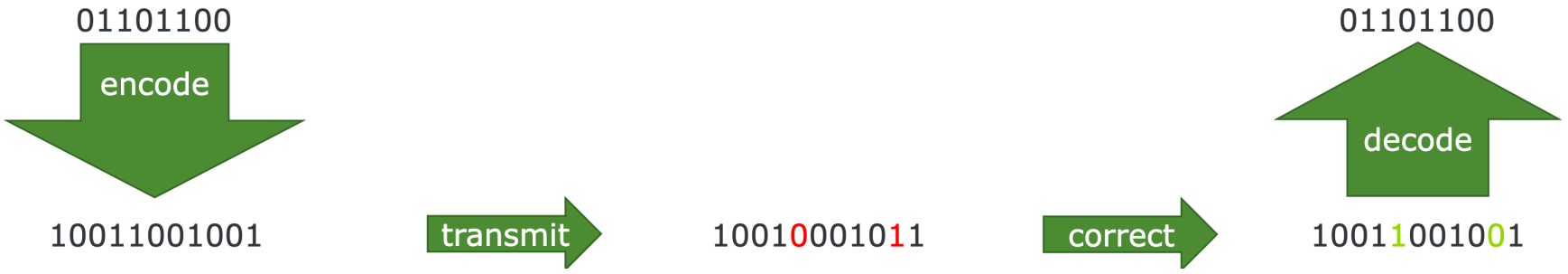
- large signatures,
- large public keys or authentication paths,
- state management issues for stateful schemes,
- slower signing/verification than classical ECDSA in many settings.

Still, for long-term authenticity, software updates, firmware signing, certificate authorities, and archival signatures, hash-based signatures are very compelling.

::.. Code-Based Cryptography

NIST's current post-quantum portfolio is mostly lattice-based for general-purpose key establishment, but code-based cryptography remains important for diversification.

Code-based cryptography is built from error-correcting codes. In communication theory, an error-correcting code allows a sender to encode a message so that the receiver can recover it even if some errors are introduced during transmission.



::.. Code-Based Cryptography

NIST's current post-quantum portfolio is mostly lattice-based for general-purpose key establishment, but code-based cryptography remains important for diversification.

Code-based cryptography is built from error-correcting codes. In communication theory, an error-correcting code allows a sender to encode a message so that the receiver can recover it even if some errors are introduced during transmission.

In cryptography, the same idea is used differently:

- add artificial errors deliberately
- and make decoding hard for anyone who does not know a secret structure.

::.. Code-Based Cryptography

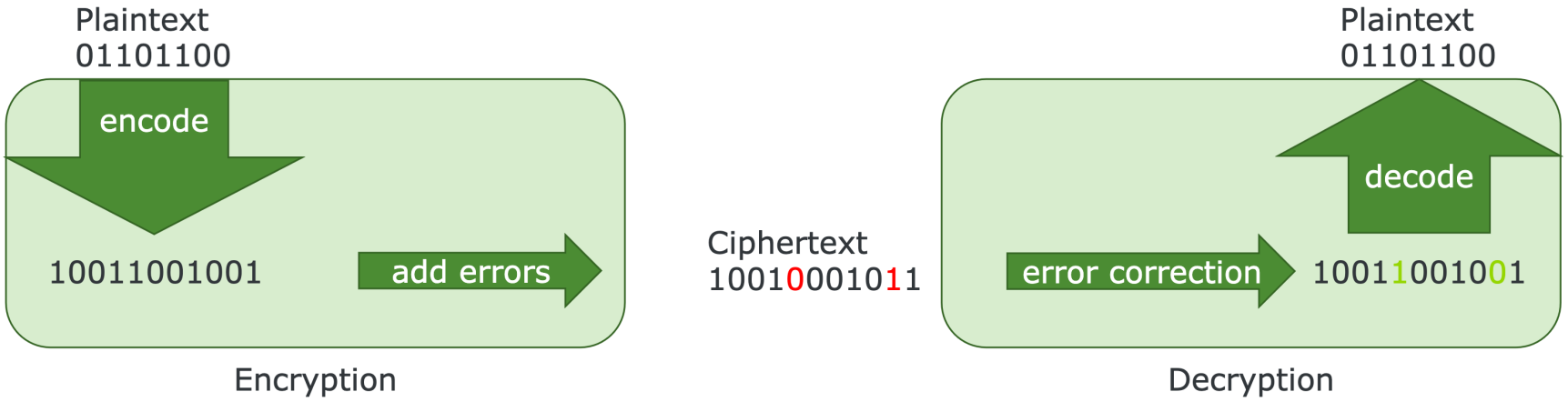
The core idea is:

- easy decoding with secret code structure,

but

- hard decoding for a random-looking code.

This is the **McEliece paradigm**.



A code-based public-key encryption scheme usually works like this plaintext⟶codeword⟶codeword + error=ciphertext.

::.. Code-Based Cryptography

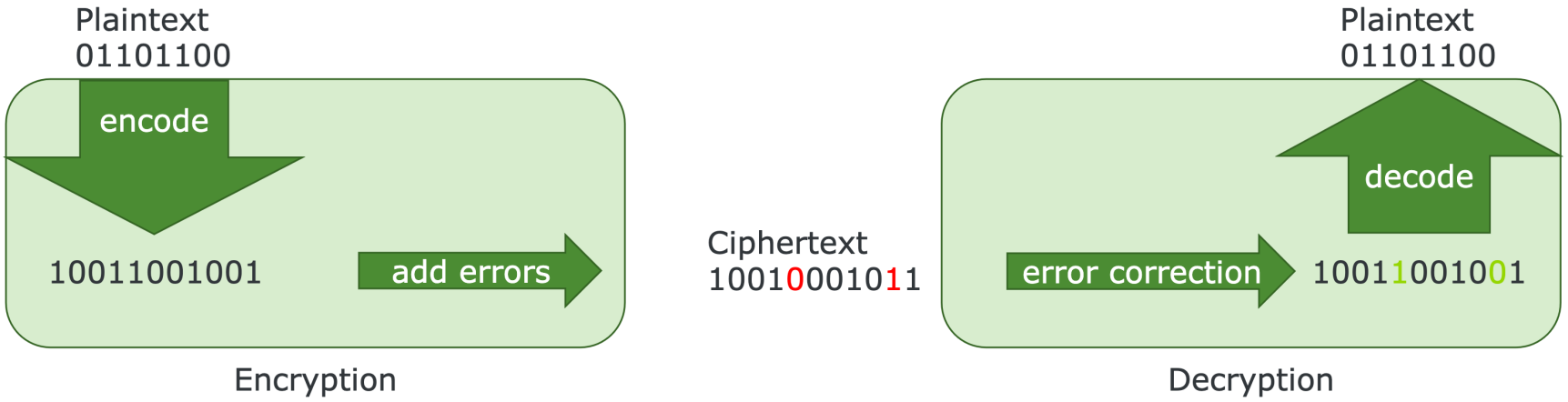
The core idea is:

- easy decoding with secret code structure,

but

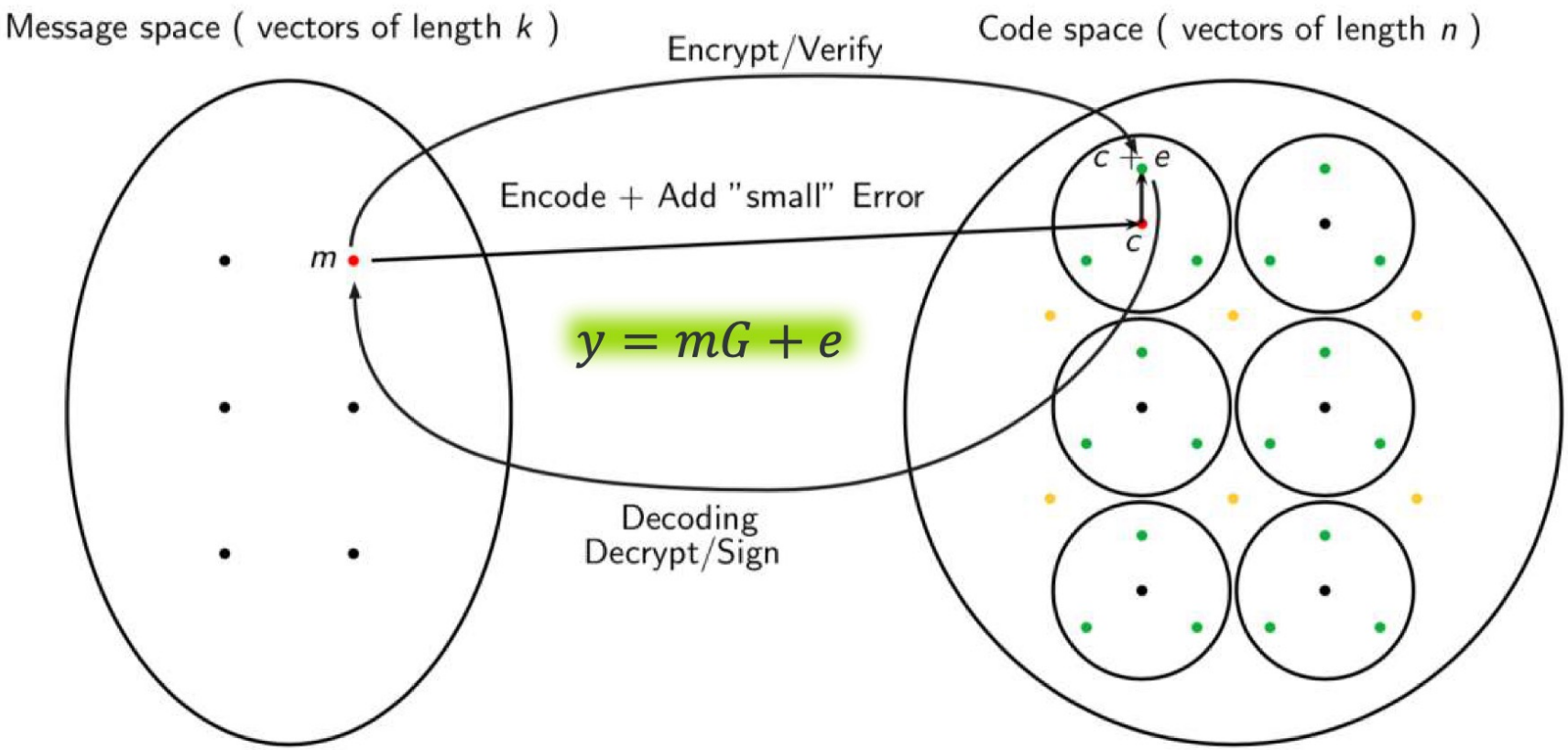
- hard decoding for a random-looking code.

This is the **McEliece paradigm**.



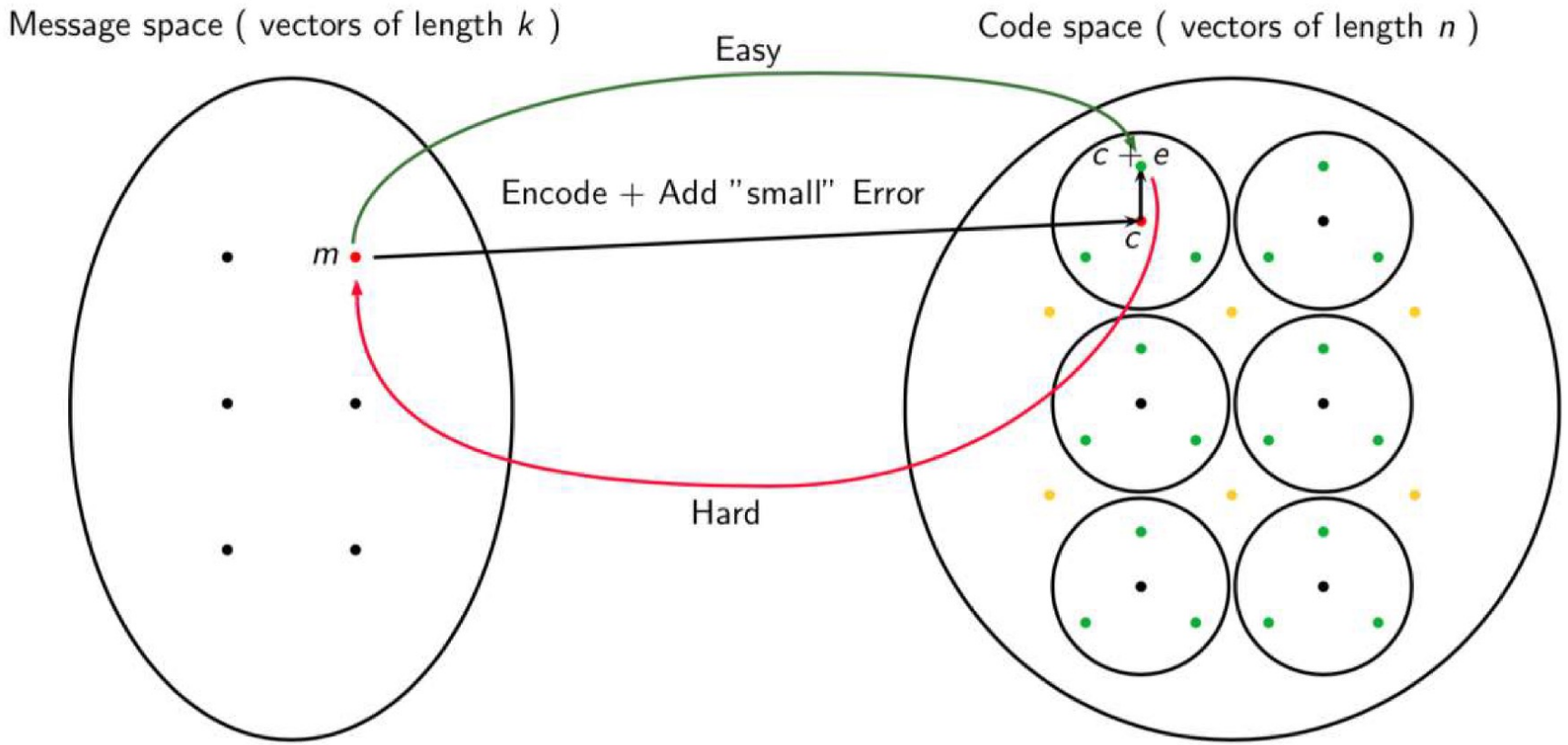
The legitimate receiver knows a hidden decoding algorithm and removes the error. The attacker sees only a random-looking linear code and must solve a hard decoding problem.

::.. Code-Based Cryptography



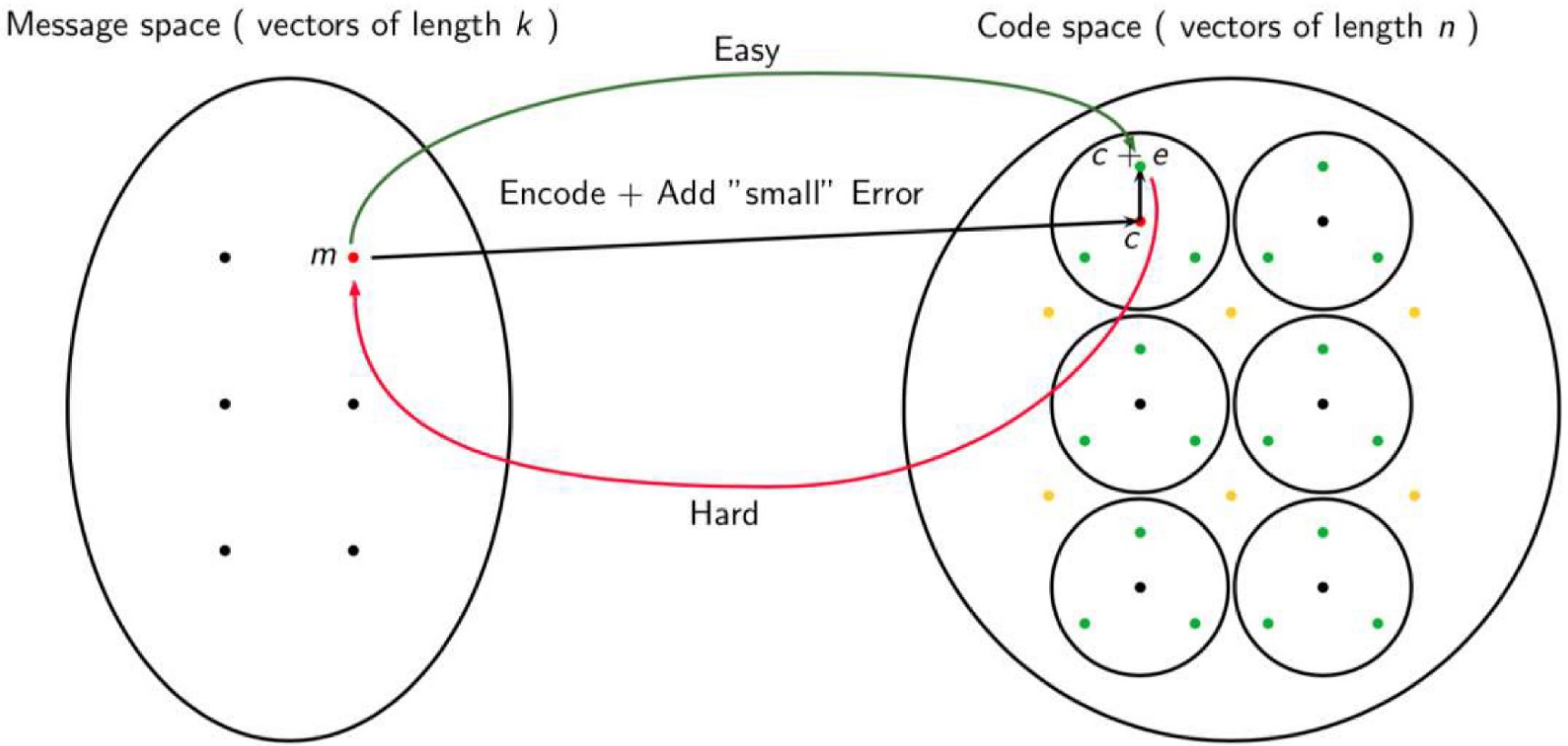
Encoding is usually done by multiplying a vector by the generator matrix G of the code, while decoding is hard in general as G is not unique, contains a base of the code space.

::.. Code-Based Cryptography



If code not random linear code, but has special algebraic structure. The encoding can be performed by $c = m \cdot (S \cdot G \cdot P)$, where G is secret and the public $S \cdot G \cdot P$ looks as the generator matrix of a random linear code, for random invertible S, P.

::.. Code-Based Cryptography



If code not random linear code, but has special algebraic structure. The decoding of $c + e$ can be done efficiently only knowing G, S, P.

::.. Code-Based Cryptography

Let $\mathbb{F}_2 = \{0,1\}$ be the binary field, Addition is XOR: $1 + 1 = 0$.

A binary linear code C of length n and dimension k is a k-dimensional subspace of $\mathbb{F}_2^n$, so $C \subseteq \mathbb{F}_2^n$, and | C |$= 2^k$. An [n,k]-linear code maps k-bit messages to n-bit codewords:

$m \in \mathbb{F}_2^n \mapsto c \in \mathbb{F}_2^n$. Usually $n > k$.

The extra n−k bits are redundancy, allowing error correction.

A linear code can be described by a generator matrix $G \in \mathbb{F}_2^{k \times n}$.A message row vector $m \in \mathbb{F}_2^k$ is encoded as $c = m \cdot G$. The code is $C = \{m \cdot G : m \in \mathbb{F}_2^k\}$. Because the code is linear, $c_1, c_2 \in C \Rightarrow c_1 + c_2 \in C$. The Hamming weight of a vector x is $wt(x) =$| $\{i : x_i \neq 0\}$ |.

::.. Code-Based Cryptography

The Hamming distance between two vectors is $d(x, y) = wt(x - y)$. Over $\mathbb{F}_2$, subtraction is the same as addition, so $d(x, y) = wt(x + y)$.

Suppose Alice sends a codeword $c = m \cdot G$. During transmission, an error vector $e \in \mathbb{F}_2^k$ is added: $r = c + e$.

If $wt(e) \leq t$, and the code can correct up to t errors, then the receiver can recover c, and hence m.

A code with minimum distance d_{min} can correct up to $t = \left\lfloor \frac{d_{min}-1}{2} \right\rfloor$ errors. In normal coding theory, errors are accidental. In McEliece cryptography, errors are deliberately inserted as the encryption mechanism.

::.. Code-Based Cryptography

The central hard problem is usually called syndrome decoding or general decoding.

- Informally: Given a random-looking linear code and a noisy vector $r = c + e$, where $c \in C$, $wt(e) = t$, recover e or c.

For a generic random linear code, this is believed to be hard, even for quantum computers. The best known attacks are variants of information-set decoding.

The decoding problem is NP-hard in general, although NP-hardness by itself does not automatically imply security for cryptographic parameter ranges. The practical security comes from decades of analysis showing that generic decoding remains expensive for carefully chosen parameters.

::.. Code-Based Cryptography

Besides a generator matrix G, a linear code can also be described by a parity-check matrix $H \in \mathbb{F}_2^{(n-k)\times n}$. A vector $c \in \mathbb{F}_2^n$ is a codeword if and only if $Hc^T = 0$.

Now suppose $r = c + e$. Compute the syndrome: $s = Hr^T$. Since c is a codeword, $Hc^T = 0$. Therefore, $s = H(c + e)^T = Hc^T + He^T = He^T$. So the syndrome depends only on the error: $s = He^T$.

Syndrome decoding asks: Given H and s, find a low-weight vector e such that $He^T = s$.

This is the form used by many McEliece-like and Niederreiter-like cryptosystems.

::.. McEliece Cryptosystems

The classic McEliece cryptosystem uses binary Goppa codes, defined using an algebraic curve over a finite field and a set of distinct evaluation points, which have efficient secret decoding algorithms but can be disguised to look like random linear codes.

Goppa codes are a specific type of Error-Correcting Code (ECC) that can be constructed using elliptic curves

The private key consists of a structured code with efficient decoding, plus scrambling transformations.

Let G be a generator matrix of a secret [n,k] code capable of correcting t errors. Alice chooses $S \in \mathbb{F}_2^{k \times k}$, an invertible scrambling matrix $P \in \mathbb{F}_2^{n \times n}$, a permutation matrix, and a structured generator matrix G.

::.. McEliece Cryptosystems

The public generator matrix is $G_{pub} = S \cdot G \cdot P$, while the private key is $(S, G, P, Decode_G)$. The public key G_{pub} should look like the generator matrix of a random linear code, while the private key holder knows how to undo the disguise and decode efficiently.

To encrypt a message $m \in \mathbb{F}_2^k$, choose a random error vector $e \in \mathbb{F}_2^k$ with $wt(e) = t$. Compute $c = m \cdot G_{pub} + e$, which is the ciphertext. The error e is not noise from the channel. It is deliberately introduced.

The receiver has G_{pub} and c, multiply by P^{-1} on the right: $c \cdot P^{-1} = m \cdot S \cdot G + e \cdot P^{-1}$.

::.. McEliece Cryptosystems

Because P is a permutation matrix, $wt(eP^{-1}) = wt(e) = t$.

So, $c \cdot P^{-1}$ is a codeword of the secret code generated by G, plus a correctable error. Using the secret decoder for G, the receiver recovers $m \cdot S$. Then multiply by S^{-1} to recover m. Thus the secret structure enables efficient decoding, while the public code appears random.

An attacker sees G_{pub} and $c = m \cdot G_{pub} + e$. They must recover m, e, or both. There are two main attack strategies.

Attack 1: Decode the random-looking code - The attacker treats G_{pub} as a generic generator matrix and tries to solve $c = m \cdot G_{pub} + e, wt(e) = t$. This is generic decoding.

::.. McEliece Cryptosystems

Attack 2: Recover the hidden structure - The attacker tries to recognize that $G_{pub} = S \cdot G \cdot P$ comes from a disguised Goppa code and recover an equivalent secret key. For well-chosen binary Goppa codes, no efficient structural attack is known.

McEliece-type systems have resisted known classical and quantum attacks.

- There is no factoring problem, no discrete logarithm, and no elliptic-curve group. This is why it is post-quantum relevant.
- Shor's algorithm attacks algebraic group problems. It does not solve generic decoding of random linear codes.
- Quantum computers can give some speedups to search-like subroutines, but they do not currently yield a Shor-like polynomial-time attack against the decoding problems.

::.. Niederreiter Cryptosystems

A closely related system is the **Niederreiter cryptosystem**.

Instead of publishing a generator matrix, it publishes a disguised parity-check matrix.

Let H be a secret parity-check matrix of a code with efficient decoding. Choose an invertible matrix $S \in \mathbb{F}_2^{(n-k)\times(n-k)}$ and a permutation matrix $\mathrm{P} \in \mathbb{F}_2^{n\times n}$.

The public parity-check matrix is $H_{pub} = S \cdot H \cdot P$.

- To encrypt, choose a low-weight error vector $e \in \mathbb{F}_2^n$, $wt(e) = t$. Compute the syndrome $s = H_{pub} \cdot e^T$, which is the ciphertext.
- To decrypt, compute $S^{-1} \cdot s = H \cdot P \cdot e^T$.

::.. Niederreiter Cryptosystems

Let $e' = e \cdot P^T$ or equivalently account for the permutation depending on row/column convention. The receiver uses the secret decoder to recover the permuted error and then applies the inverse permutation to recover e.

In Niederreiter encryption, the “message” is not an arbitrary vector m multiplied by a generator matrix, as in McEliece. Instead, the message is encoded as the error vector e. The receiver obtained th original message by recovering e from its syndrome s.

In modern KEMs, the recovered error vector e is used to derive a shared key.

::.. Discussion on Code-Based Encrypt.

The original McEliece encryption scheme is not automatically secure against adaptive chosen-ciphertext attacks. Modern KEMs need strong security notions such as IND-CCA security: an attacker should not learn the shared key even if they can query a decapsulation oracle on other ciphertexts.

Modern McEliece-like KEMs therefore use transformations similar in spirit to Fujisaki–Okamoto-style techniques:

- randomness is derived/checked,
- invalid ciphertexts are rejected safely,
- shared keys are hashed with ciphertext context,
- decapsulation must avoid leakage.

The KEM design must ensure that malformed ciphertexts do not reveal information about the secret decoder.

::.. Discussion on Code-Based Encrypt.

Some code-based schemes have a possibility of decryption failure, even for honestly generated ciphertexts, with very small probability. For cryptographic protocols, decryption failures are dangerous because they can sometimes be exploited in attacks.

Classic McEliece using Goppa decoding is designed around exact correction up to t errors, so properly formed ciphertexts are decoded reliably. For other schemes, decryption-failure probability must be tightly analyzed.

Code-based cryptography is mainly used for public-key encryption and key encapsulation. It is not mainly used for practical digital signatures. McEliece-like systems are usually discussed as replacements for RSA encryption or ECDH-style key establishment, not as replacements for ECDSA.

::.. Discussion on Code-Based Sign.

The basic syndrome function is $f_H(e) = He^T \in \mathbb{F}_2^r$. The one-way problem is:

- Given H and $y \in \mathbb{F}_2^r$, find a low-weight $e \in \mathbb{F}_2^n$ such that $H \cdot e^T = y, wt(e) = t$.

This is the syndrome decoding problem.

The function $e \mapsto H \cdot e^T$ is easy to compute but hard to invert for a random-looking H, provided e is required to have small Hamming weight.

This is the code-based analogue of the RSA one-way function $x \mapsto x^e mod N$.

For RSA signatures, the trapdoor is knowledge of the private exponent d. For code-based hash-and-sign, the trapdoor is knowledge of a secret decoding algorithm for the hidden code.

::.. Discussion on Code-Based Sign.

Signatures are harder because a signature usually requires inverting a public one-way function on a hash value.

For RSA: $\sigma = H(M)^d mod N$, and verification checks $\sigma^e \equiv H(M)(modN)$. The public hash value $H(M)$ is always in the image of the RSA permutation, assuming the padding is well-formed. For syndrome decoding, this is not automatically true.

A random syndrome $y \in \mathbb{F}_2^r$ may not correspond to a decodable error of weight exactly t, or may be hard to decode even for the trapdoor decoder if the code only corrects specific error patterns. That is the fundamental difficulty behind code-based hash-and-sign signatures.

::.. Discussion on Code-Based Sign.

Let's see this hash-and-sign approach with the canonical historical scheme is **CFS**, by Courtois, Finiasz, and Sendrier:

- Let H_{sec} be the parity-check matrix of a secret code with an efficient decoding algorithm capable of decoding errors of weight at most t. In CFS-like schemes this can be a high-rate binary Goppa code.
- Choose: $S \in \mathbb{F}_2^{r\times r}$ invertible, and $P \in \mathbb{F}_2^{n\times n}$ a permutation matrix. Publish $H_{pub} = S \cdot H_{sec} \cdot P$, whihc is the public key pk. The private key is $sk = (S, H_{sec}, P, Decode_{H_{sec}})$.
- Given a message M the signer computes a hash-derived syndrome $y = H_{hash}(M) \in \mathbb{F}_2^r$.
- The signer wants to find $e \in \mathbb{F}_2^n$ such that $H_{pub} \cdot e^T = y$ and $wt(e) = t$. If such an e is found, the signature is $\sigma = e$.

::.. Discussion on Code-Based Sign.

- Verification checks: $H_{pub} \cdot \sigma^T =? H_{hash}(M)$, and $wt(\sigma) = t$. This is directly analogous to RSA verification: $\sigma^e \equiv H(M)(modN)$.

The crucial issue is that the decoder may not be able to decode every syndrome. The total number of possible syndromes is 2^r. A random syndrome is decodable with probability approximately $p \approx \frac{\binom{n}{t}}{2^r}$. For many codes and parameters, this probability is tiny. So if the signer hashes directly to $y = H(M)$, then y may not be decodable.

CFS solves this by adding a counter or salt: $y_i = H(M \parallel i)$. The signer tries $i = 0,1,2, \ldots$ until finding a decodable syndrome. Then the signature is $\sigma = (e, i)$.

::.. Discussion on Code-Based Sign.

Verification recomputes $y_i = H(M \parallel i)$ and checks $H_{pub} \cdot e^T = y_i$, $wt(e) = t$. So the real signing algorithm is probabilistic and may require many trials.

The probability $p \approx \frac{\binom{n}{t}}{2^r}$ must not be astronomically small, or signing becomes infeasible. To increase this probability, CFS uses high-rate codes, meaning $k \approx n$, so $r = n - k$ is relatively small.

That reduces the syndrome space 2^r.

But high-rate Goppa codes create another problem: they require very large public keys and may expose distinguishers or structural issues.

This is why CFS-like signatures tend to have short signatures but very large public keys and sometimes slow signing.

::.. Discussion on Code-Based Sign.

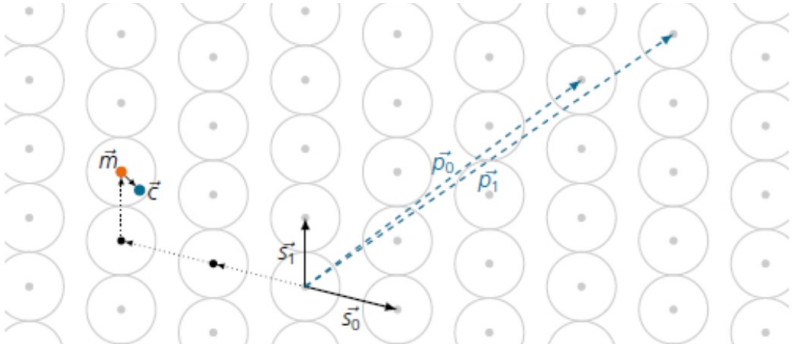
A second approach avoids a trapdoor decoder.

- Instead of building a signature by inverting $e \mapsto H \cdot e^T$, one builds an identification protocol proving knowledge of a secret low-weight vector e such that $H \cdot e^T = s$. Then the **Fiat-Shamir transform** converts this into a signature scheme.

This is analogous to **Schnorr signatures**: use the **Stern's identification** protocol, which becomes a signature by replacing the verifier's random challenge with a hash.

Property	**Trapdoor hash-and-sign**	**Stern/Schnorr-like Fiat–Shamir**
Main idea	invert syndrome using decoder	prove knowledge of low-weight preimage
Trapdoor?	yes	no
Private key	secret decoder	low-weight vector e
Public key	scrambled code matrix	(H, s), often seed-compressed
Signature	e plus salt/counter	Fiat–Shamir proof transcript
Signature size	short	large
Public key size	large	small
Main hard problem	syndrome decoding + code hiding	syndrome decoding
Main issue	hash may not be decodable	large transcripts; quantum FS subtleties

::.. Introduction to Lattice



Lattice-based cryptography relies on the computational hardness of finding the shortest vector in a high-dimensional lattice.

An n-dimensional lattice is a regular, repeating grid of points in n-dimensional Euclidean space ($\mathbb{R}^n$). Formally, it is defined as points made of a set of linearly independent basis vectors $v_1, \ldots, v_n \in \mathbb{R}^n$ and a set of integers $a_1, \ldots, a_n \in \mathbb{R}^n$:

$$p = \sum_{i=1}^{n} a_i v_i$$

The lattice is a set of points and is not bounded to a specific basis, as the same set of points can be generated by many different bases.

::.. Introduction to Lattice

Let $b_1, b_2, \ldots, b_n \in \mathbb{R}^n$ be linearly independent vectors, the lattice generated by them is $\mathcal{L}(B) = \{z_1 b_1 + z_2 b_2 + \cdots + z_n b_n : z_i \in \mathbb{Z}\}$. A **good basis** has vectors that are short, nearly orthogonal, well-conditioned. A **bad basis** has vectors that are long, nearly parallel or skewed, poorly conditioned.

A common way to measure basis quality is using **Gram-Schmidt orthogonalization**. Given basis vectors $b_0, b_1, \ldots, b_n$, Gram-Schmidt produces orthogonalized vectors $b_0^*, b_1^*, \ldots, b_n^*$:

- If the original basis is nearly orthogonal, then the Gram-Schmidt vectors are not much shorter than the original vectors.
- If the basis is bad, some Gram-Schmidt vectors become very short, indicating strong linear dependence.

::.. Introduction to Lattice

Cryptography exploits this gap:

- good basis implies easy computations, and represents the secret basis,
- Bad/random-looking basis implies hard computations, and represents the public basis which describes the same lattice, but in an awkward way.

Lattice problems are a class of highly complex computational tasks based on lattices. Because finding exact answers in high dimensions is widely believed to be impossible in polynomial time, they serve as the mathematical bedrock for post-quantum cryptography.

::.. Introduction to Lattice

The **Shortest Vector Problem** (SVP) asks:

- given a basis B, find a nonzero vector $v \in \mathcal{L}(B) \setminus \{0\}$ with minimal Euclidean norm: $\| v \| = \min_{u \in \mathcal{L}(B) \setminus \{0\}} \| u \|$.

The exact problem is very hard. Cryptography usually relies on approximate versions: $\| v \| \leq \gamma \lambda_1(\mathcal{L})$, where $\lambda_1(\mathcal{L})$ is the length of the shortest nonzero lattice vector, and γ is an approximation factor.

The **Closest Vector Problem** (CVP) asks:

- given a lattice $\mathcal{L}$ and a target point $t \in \mathbb{R}^n$, find $v \in \mathcal{L}$ minimizing $\| t - v \|$.

This is also hard in high dimension.

::.. Introduction to Lattice

The **Short Integer Solution** problem (SIS) is :

- given a random matrix $A \in \mathbb{Z}_q^{n \times m}$ find a nonzero short vector $z \in \mathbb{Z}^m$ such that $Az = 0(modq)$.

The vector z must be short, for example with $\| z \| \leq \beta$. Without the shortness condition, solving $Az = 0(modq)$ is easy linear algebra, but the hardness comes from requiring a short nonzero solution.

The **Learning With Errors** problem (LWE) is probably the most important problem :

- let q be a modulus, $A \in \mathbb{Z}_q^{m \times n}$ be public and uniformly random, $s \in \mathbb{Z}_q^n$ be a secret vector, $e \in \mathbb{Z}^m$ be a small error vector, and the public value is $b = As + e(modq)$.

::.. Introduction to Lattice

- given (A, b), recover s, or distinguish $b = As + e$ from a uniformly random vector in $\mathbb{Z}_q^n$.

Without the error term e, this is just linear algebra, Gaussian elimination and with enough equations, one solves for s efficiently. But with small errors, the problem becomes computationally hard. This is the core trick.

Lattice-based cryptography is often “linear algebra plus noise.”

- The legitimate user knows the secret and can tolerate or remove small noise. The attacker sees noisy equations and cannot solve them efficiently. So the error vector e plays a role similar to the artificial error vector in code-based cryptography. In both cases, small noise/errors hide linear structure.

::.. Lattice-Based Cryptography

A simplified public-key encryption scheme based on LWE is named after the **Regev-style idea** and works as follows:

- Let $A \in \mathbb{Z}_q^{m \times n}$ be public, the secret key is $s \in \mathbb{Z}_q^n$. The public key is $b = As + e(modq)$, where e is small.
- To encrypt a bit $\mu \in \{0,1\}$, choose a small/random binary vector $r \in \{0,1\}^m$. Compute $u = A^T r(modq)$, $v = b^T r + \mu \left\lfloor \frac{q}{2} \right\rfloor (modq)$. The ciphertext is (u, v).
- Now decrypt using s: $v - s^T u = b^T r + \mu \left\lfloor \frac{q}{2} \right\rfloor - s^T A^T r$. Since $s^T A^T r = (As)^T r$, we get $v - s^T u = (As + e)^T r + \mu \left\lfloor \frac{q}{2} \right\rfloor - (As)^T r$. So $v - s^T u = e^T r + \mu \left\lfloor \frac{q}{2} \right\rfloor (modq)$. Because e and r are small, the term $e^T r$ is small. Therefore the decryptor sees a value close to either 0 or $^q/_2$.

::.. Lattice-Based Cryptography

- If it is close to 0, output $\mu = 0$.
- If it is close to $q/2$, output $\mu = 1$.

So correctness depends on the noise being small enough:

$|e^T r| \ll q/4$. Security depends on the attacker being unable to distinguish $b = As + e$ from random.

Modern protocols usually do not use raw public-key encryption. They use a key encapsulation mechanism, or KEM.

In a lattice-based KEM, encapsulation produces a ciphertext C containing noisy linear-algebraic data. The decapsulator uses the secret key to recover a shared secret or seed and derives $K = KDF(shared\ secret, C, context)$. The standardized lattice-based KEM is ML-KEM, derived from Kyber.

::.. Lattice-Based Cryptography

Lattice cryptography also gives digital signatures.

- The main standardized lattice-based signature is ML-DSA, derived from CRYSTALS-Dilithium and standardized as FIPS 204. NIST says ML-DSA is intended for generating and verifying digital signatures and is believed secure even against large-scale quantum adversaries.

The main hard problem behind lattice signatures is usually related to SIS or Module-SIS. Lattice signatures use a secret trapdoor or secret short vectors to produce short valid responses. The public verification equation is linear modulo q, but the signature must be short.

- A simplified signature is $Az = c(modq)$, with $\| z \| \leq B$.
- Verification checks both $Az = c(modq)$ and z is short.

The shortness is the key, without it, solving the equation is easy.

::.. Lattice-Based Cryptography

Many lattice signatures use a paradigm related to identification protocols and the Fiat–Shamir transform.

- A very simplified pattern is that the signer commits to a masked lattice value: $w = Ay$. The challenge is derived by hashing $c = H(\mu, w)$, where μ is the message digest.
- The signer responds with something like $z = y + cs$.
- The verifier checks that z is short and that the reconstructed commitment is consistent $Az - ct \approx w$. Here s is secret, $t = As + e$ is public, and "$\approx$" reflects rounding/high-bit extraction.

The danger is that z may leak information about s. To avoid this, lattice signatures carefully sample y, bound z, and sometimes reject and restart. This is the "with aborts" idea.

::.. Lattice-Based Cryptography

Some lattice encryption/KEM schemes have a small probability of decryption failure.

- A failure happens if the total noise is too large and the receiver decodes the wrong message.

Modern schemes choose parameters so that failure probability is negligible or effectively eliminated for valid ciphertexts. They also use CCA-secure transformations so attackers cannot exploit failures.

Raw LWE encryption is not automatically secure against adaptive chosen-ciphertext attacks. Modern KEMs need IND-CCA security.

So modern lattice KEMs are carefully engineered CCA-secure constructions built from LWE-like primitives.

::.. Lattice-Based vs Code-Based

- Both use noisy linear algebra and hide information using an error/noise term.
- But the underlying hard problems differ:
  - code-based: decode random linear codes,
  - lattice-based: solve noisy modular equations / find short vectors.
- Practical trade-off:
  - code-based McEliece: huge public keys, conservative security,
  - lattice-based ML-KEM: compact keys, fast, widely deployable.

This is why lattice-based schemes have become the primary candidates for PQC deployment.

::.. Lattice-Based vs Hash-Based

- Hash-based signatures rely on one-way hash functions. They are conservative but often have larger signatures and limited signing structures.
- Lattice-based signatures rely on module-SIS / module-LWE / NTRU lattice assumptions. They are more compact and efficient for many applications.

So:

- SLH-DSA: conservative hash-based backup,
- ML-DSA: primary practical signature standard.

NIST's finalized standards include ML-KEM, ML-DSA, and SLH-DSA, with ML-KEM and ML-DSA being lattice-based and SLH-DSA being hash-based.